

TECH FORCE: GALACTIC
ARCHITECTURE

BINÁRIOS LINUX



6

LISTA DE FIGURAS

Figura 1 – Estrutura de objeto ELF	14
Figura 2 – Tipos de dados seu tamanho e alinhamento padrão	15
Figura 3 – Cabeçalho ELF	15
Figura 4 – Exemplo de Cabeçalho ELF.....	18
Figura 5 – Entry Point.....	18
Figura 6 – Cabeçalho ELF32 e ELF64	19
Figura 7 – Exemplo de tabela de cabeçalho de programa	20
Figura 8 – Tabela de seção.....	22
Figura 9 – Tabela de símbolos e strings	26
Figura 10 – Tabela de índice e strings	27
Figura 11 – Estrutura arquivo COFF	29
Figura 12 – Estrutura da tabela de símbolos do COFF.....	34
Figura 13 – Fluxo de compilação e linking	44
Figura 14 – Layouts.....	58

LISTA DE TABELAS

Tabela 1 – Campos do cabeçalho ELF	17
Tabela 2 – Cabeçalho do Programa.....	20
Tabela 3 – Campos da tabela de seção.....	22
Tabela 4 – Campos do file header do COFF.....	30
Tabela 5 – Flags do file header do COFF	30
Tabela 6 – Campos tabela de seção.....	31
Tabela 7 – Flags para tabela de seção	32
Tabela 8 – Realocação	33
Tabela 9 – Campos da tabela de símbolos	35
Tabela 10 – Campos do cabeçalho de seção	35

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Substituição simples	45
Código-fonte 2 – Resultado pré-processamento	45
Código-fonte 3 – Macros	46
Código-fonte 4 – Programa print RAX.....	57
Código-fonte 5 – strace cat	63
Código-fonte 6 – objdump	65
Código-fonte 7 – Script simples	72

EXEMPLO

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Criando um arquivo objeto e linkando	48
Comando de prompt 2 – Compilando GCC.....	49
Comando de prompt 3 – Usando gcc.....	50
Comando de prompt 4 – Compilando arquivo objeto	50
Comando de prompt 5 – Compilando arquivo c	50
Comando de prompt 6 – Opções de compilação	50
Comando de prompt 7 – Múltiplos arquivos	51
Comando de prompt 8 – Biblioteca	51
Comando de prompt 9 – Listando arquivos.....	51
Comando de prompt 10 – Tornando o programa executável	52
Comando de prompt 11 – Executando o binário 1	52
Comando de prompt 12 – Executando o binário 2	52
Comando de prompt 13 – PATH	52
Comando de prompt 14 – Executando programa com argumentos.....	53
Comando de prompt 15 – Direcionando a entrada	53
Comando de prompt 16 – Redirecionando o output.....	53
Comando de prompt 17 – Execução em segundo plano.....	53
Comando de prompt 18 – 16 GDB	55
Comando de prompt 19 – Importando o programa	55
Comando de prompt 20 – Configurando o disassembly.....	56
Comando de prompt 21 – Exemplo 1.....	58
Comando de prompt 22 – Breakpoint.....	58
Comando de prompt 23 – Print de caracteres.....	59
Comando de prompt 24 – Desassemble uma instrução.....	59
Comando de prompt 25 – Desassemble instrução atual.....	59
Comando de prompt 26 – Verificando o conteúdo de codes.....	59
Comando de prompt 27 – Examinando os 8 bytes	60
Comando de prompt 28 – Segundo qword.....	60
Comando de prompt 29 – Layou src	60
Comando de prompt 30 – backtrace	60
Comando de prompt 31 – strace	62
Comando de prompt 32 – Instalando um arquivo deb.....	69
Comando de prompt 33 – Instalando dependências	69
Comando de prompt 34 – Removendo um pacote deb.....	69
Comando de prompt 35 – Criando um pacote	70
Comando de prompt 36 – Criando o pacote de controle.....	70
Comando de prompt 37 – Gerando o deb.....	70
Comando de prompt 38 – Instalando rpm	71
Comando de prompt 39 – Instalando dependências	71
Comando de prompt 40 – Removendo pacotes	71
Comando de prompt 41 – Gerando pacotes rpm	72

SUMÁRIO

BINÁRIOS LINUX.....	8
1 INTRODUÇÃO AOS BINÁRIOS LINUX.....	8
2 DEFINIÇÃO.....	9
3 DIFERENTES TIPOS DE BINÁRIOS.....	10
4 COMPREENDENDO O FORMATO DE ARQUIVOS BINÁRIOS	11
4.1 Formato ELF	12
4.1.1 Representação de dados	14
4.1.2 Cabeçalho ELF.....	15
4.1.3 Cabeçalho do Programa (Phdr).....	18
4.1.4 Tabela de Seções (Shdr)	21
4.1.5 Tabela de String e Símbolos	26
4.1.6 Tabela de realocação (Rel e Rela).....	27
4.1.7 Dynamic tags (Dyn).....	27
4.1.8 Notes (Nhdr).....	27
4.2 Formato COFF	28
4.2.1 Estrutura do COFF	28
4.2.2 File Header.....	29
4.2.3 Estrutura da tabela de seção	30
4.2.4 Seção de realocação.....	32
4.2.5 Estrutura da tabela de símbolos e conteúdo	34
4.2.6 Tabelas de strings	35
5 COMPATIBILIDADE DE BINÁRIOS.....	36
5.1 Definição e importância da compatibilidade de binários.....	36
5.2 Compatibilidade ABI.....	36
5.3 Interface do <i>system call</i>	38
5.3.1 Como as chamadas de sistema funcionam.....	38
5.4 Compatibilidade de bibliotecas.....	39
5.4.1 Compatibilidade binária de biblioteca.....	40
5.4.2 Compatibilidade de código-fonte de bibliotecas	41
6 CRIANDO E EXECUTANDO BINÁRIOS	42
6.1 Compilação e linking	43
6.1.1 Pré-processador.....	45
6.1.2 Tradução	47
6.1.3 Linking	48
6.1.4 Explicando gcc	49
6.2 Executando um arquivo binário	51
7 ANÁLISE E DEBUGGING DE BINÁRIOS.....	54
7.1 Introdução ao gdb	55
7.2 strace	60
7.3 Objdump e readelf.....	63
7.3.1 Objdump.....	63
7.3.2 Readelf.....	65
8 PACOTES E DISTRIBUIÇÃO DE BINÁRIOS	66

8.1 Bibliotecas estáticas e dinâmicas.....	66
8.2 Criando .deb e .rpm.....	68
8.2.1 .deb	68
8.2.2 .rpm	70
REFERÊNCIAS.....	73

EMANIP

BINÁRIOS LINUX

1 INTRODUÇÃO AOS BINÁRIOS LINUX

Os binários desempenham um papel fundamental no sistema operacional Linux, permitindo a execução de programas e aplicativos. Neste capítulo, exploraremos os aspectos essenciais dos binários Linux, abrangendo sua definição, propósito e tipos diferentes. Além disso, mergulharemos na compreensão do formato de arquivos binários, destacando o formato ELF (*Executable and Linkable Format*) e o COFF (*Common Object File Format*), e faremos uma comparação entre eles.

A compatibilidade de binários é outro tópico crucial a ser abordado. Exploraremos a definição e importância da compatibilidade de binários, juntamente com o conceito de ABI (*Application Binary Interface*) e a relevância da compatibilidade reversa. Compreenderemos como criar e executar binários, introduzindo o processo de compilação e vinculação, explicando o papel do GCC (*GNU Compiler Collection*) e discutindo as permissões necessárias para a execução de arquivos binários.

No campo da análise e depuração de binários, exploraremos ferramentas essenciais como GDB (*GNU Debugger*), LLDB (*LLVM Debugger*) e strace. Também aprenderemos sobre a análise de binários usando objdump e readelf, além de mergulharmos na engenharia reversa de binários.

A distribuição de binários é um aspecto importante do ecossistema Linux, e abordaremos os conceitos de link estático e dinâmico, bem como as bibliotecas compartilhadas. Discutiremos a criação de pacotes .deb e .rpm, que facilitam a distribuição e instalação de binários.

Por fim, examinaremos os binários do sistema e as ferramentas essenciais do Linux, como ls, cd, mv e rm, e explicaremos a estrutura de diretórios em /bin e /sbin. Também discutiremos a variável PATH e sua influência na localização e execução de binários.

Para concluir, este capítulo fornecerá uma base sólida para compreender os binários Linux, desde sua criação e execução até sua análise e distribuição. Esses

conhecimentos essenciais nos permitirão explorar ainda mais os aspectos avançados desse ecossistema poderoso e versátil.

2 DEFINIÇÃO

Um arquivo binário é um arquivo cujo conteúdo está no formato binário, consistindo de uma sequência de bytes sequenciais, sendo cada byte composto por oito bits. Esses arquivos podem conter código-fonte compilado (código de máquina) e também podem ser chamados de executáveis.

Adicionalmente, os arquivos binários não podem ser lidos diretamente por seres humanos e requerem um programa especializado capaz de interpretar esses dados. Somente assim, uma instrução codificada em binário pode ser compreendida e processada adequadamente.

Geralmente, um arquivo binário inclui um cabeçalho que indica o tipo do arquivo e fornece informações adicionais sobre seu conteúdo. Esse cabeçalho pode conter alguns caracteres legíveis por humanos, mas, em geral, os arquivos binários como um todo necessitam de softwares específicos ou hardwares para serem lidos e processados.

Os arquivos binários são amplamente utilizados no desenvolvimento de aplicativos e outros tipos de software. No entanto, os desenvolvedores não interagem diretamente com esses arquivos. Em vez disso, eles utilizam linguagens de programação de alto nível para desenvolver suas aplicações, que são posteriormente compiladas em binários executáveis.

No contexto do sistema operacional GNU/Linux, os binários desempenham um papel fundamental e podem ser encontrados em diferentes diretórios conforme veremos a seguir.

3 DIFERENTES TIPOS DE BINÁRIOS

No sistema operacional Linux, existem diferentes tipos de binários, cada um com uma finalidade específica. Vamos explorar os principais tipos de binários encontrados no Linux:

- **Binários do sistema (/bin e /sbin):**

Os diretórios /bin (binários do sistema) e /sbin (binários do sistema de administração) contêm os binários essenciais para o funcionamento básico do sistema operacional. Esses binários são de uso geral e estão disponíveis para todos os usuários. Eles executam comandos e fornecem funcionalidades essenciais do sistema, como manipulação de arquivos, gerenciamento de processos, manipulação de rede, entre outros. Exemplos de binários encontrados nesses diretórios são: ls, cp, mv, rm, cat, chmod, mount, umount, entre outros.

- **Bibliotecas compartilhadas (/lib e /lib64):**

As bibliotecas compartilhadas são arquivos binários que contêm código e recursos comuns utilizados por vários programas. Essas bibliotecas são compartilhadas entre diferentes aplicativos, economizando espaço em disco e permitindo a reutilização de código. No Linux, os diretórios /lib (ou /lib64 em sistemas de 64 bits) contêm as bibliotecas compartilhadas necessárias para a execução dos programas instalados no sistema. Exemplos de bibliotecas compartilhadas são: libc, libpthread, libssl, libcrypto, entre outras.

- **Binários do usuário (/usr/bin e /usr/sbin):**

Os diretórios /usr/bin (binários do usuário) e /usr/sbin (binários do usuário de administração) contêm binários adicionais instalados por usuários ou pacotes de software. Esses binários são específicos para cada usuário ou programa e não fazem parte do sistema operacional base. Os binários do usuário podem ser aplicativos, utilitários ou scripts personalizados. Por exemplo, um usuário pode instalar um editor de texto específico ou um programa de reprodução de mídia e os binários correspondentes ficarão localizados nesses diretórios.

- **Binários do ambiente gráfico (/usr/bin e /usr/sbin):**

Em sistemas com ambientes gráficos, como GNOME ou KDE, os binários relacionados ao ambiente de desktop são encontrados nos diretórios `/usr/bin` e `/usr/sbin`. Esses binários estão associados a aplicativos gráficos, como navegadores de internet, editores de imagem, gerenciadores de arquivos e configurações do sistema gráfico.

- **Binários de terceiros (/opt):**

O diretório `/opt` é usado para instalar binários de aplicativos de terceiros no sistema Linux. Esses binários são geralmente fornecidos por fornecedores de software externos ao sistema operacional e podem incluir aplicativos comerciais ou proprietários. Dentro do diretório `/opt`, cada aplicativo possui seu próprio subdiretório, no qual estão localizados os binários específicos daquele software.

Além desses tipos de binários, existem também os módulos de kernel, que são binários específicos do núcleo do sistema operacional Linux. Esses módulos fornecem funcionalidades adicionais ao kernel, como drivers de dispositivo ou suporte para sistemas de arquivos específicos.

Cada tipo de binário no Linux desempenha um papel fundamental no funcionamento do sistema operacional e na execução de aplicativos. Eles fornecem as funcionalidades necessárias para o usuário interagir com o sistema, executar tarefas específicas e aproveitar as capacidades do software instalado.

4 COMPREENDENDO O FORMATO DE ARQUIVOS BINÁRIOS

O formato de arquivo binário é uma representação compacta dos dados armazenados em um arquivo, onde as informações são codificadas em uma sequência de bytes. Ao contrário de formatos de arquivo de texto, que podem ser lidos e interpretados diretamente por humanos, os arquivos binários requerem um programa especializado para entender sua estrutura e conteúdo.

Os arquivos binários são amplamente utilizados no desenvolvimento de software, pois permitem armazenar e executar código de máquina de forma eficiente. Esses arquivos podem conter instruções de código compilado, dados, bibliotecas compartilhadas e outros recursos necessários para a execução de um programa.

No contexto do sistema operacional Linux e de outros sistemas Unix-like, dois dos formatos de arquivo binário mais comumente encontrados são o ELF (*Executable and Linkable Format*) e o COFF (*Common Object File Format*). Esses formatos desempenham um papel crucial no armazenamento e na execução de programas.

O formato ELF é amplamente adotado em sistemas Unix-like, incluindo Linux. Ele oferece suporte a executáveis, bibliotecas compartilhadas, objetos relocáveis e outros tipos de arquivos. O ELF possui uma estrutura flexível que permite a inclusão de metadados detalhados sobre o programa, como tabelas de símbolos, seções, informações de depuração e muito mais. Essas informações são essenciais para o carregamento, a ligação e a execução corretos de um programa.

Já o formato COFF foi originalmente desenvolvido pela AT&T para uso em sistemas Unix. Embora tenha sido amplamente substituído pelo ELF em muitos ambientes, ainda é utilizado em alguns sistemas operacionais e compiladores. O COFF também é usado como formato intermediário em processos de compilação e ligação de código-fonte.

Tanto o formato ELF quanto o formato COFF possuem especificações detalhadas que definem sua estrutura interna e a maneira como os dados são organizados. Essas especificações permitem que o sistema operacional e as ferramentas de desenvolvimento entendam e manipulem os arquivos binários corretamente.

Nos próximos tópicos, exploraremos com mais detalhes o formato ELF e o formato COFF, incluindo suas características, uso e importância no desenvolvimento de software no contexto do sistema operacional Linux.

4.1 Formato ELF

ELF é o formato binário padrão em sistemas operacionais como o Linux. Algumas das capacidades do ELF incluem o linking dinâmico, o carregamento dinâmico, o controle em tempo de execução de um programa e um método aprimorado para criação de bibliotecas compartilhadas. A representação ELF dos dados de controle em um arquivo objeto é independente da plataforma, o que é uma melhoria adicional em relação a formatos binários anteriores.

A representação ELF permite que os arquivos objeto sejam identificados, analisados e interpretados de forma semelhante, tornando os arquivos objeto ELF compatíveis em várias plataformas e arquiteturas de diferentes tamanhos. Os três principais tipos de arquivos ELF são:

- **Executável:** guarda um programa que pode ser executável; o arquivo específico como o `exec` cria uma imagem de processo no sistema.
- **Relocável:** guarda o código e dados usados para fazer o link entre outros objetos para criar um executável ou objeto compartilhado
- **Objeto compartilhado:** contém o código e dados usados para fazer o link em dois contextos. Primeiro, o link que o editor enxerga `ld(SD_CMD)`. Segundo, o link dinâmico combina ele com um arquivo executável e outro objeto compartilhado para criar uma imagem de processo (carregar o programa na memória e alocar recursos para que ele seja executado).

Se você não entendeu muito bem, fique tranquilo que explicaremos mais abaixo.

Esses tipos de arquivos contêm o código, os dados e as informações sobre o programa que o sistema operacional e o editor de ligação precisam para realizar as ações apropriadas nesses arquivos.

Um executável no formato ELF consiste em um cabeçalho ELF (`<elf.h>`), seguido por uma tabela de cabeçalho do programa (*program header*) ou uma tabela de cabeçalho de seção (*section header*), ou ambas. O cabeçalho ELF sempre estará localizado no deslocamento (offset) zero do arquivo. Já a tabela de cabeçalho do programa e a tabela de cabeçalho de seção terão seus deslocamentos definidos pelo cabeçalho ELF. Essas duas tabelas descrevem as características específicas do arquivo.

Object files participam no processo de linking de programas (processo pelo qual um programa passa ao ser construído/compilado) e execução de programas. Para conveniência e eficiência, o formato de um arquivo objeto fornece uma visão paralela do conteúdo de um arquivo. A figura “Estrutura de objeto ELF” fornece uma visualização da organização da estrutura dos objetos ELF.

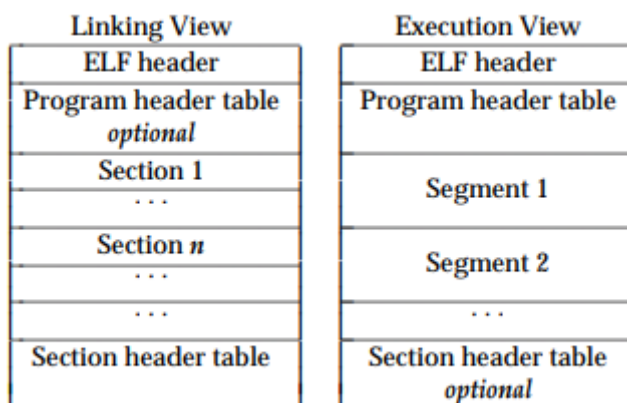


Figura 1 – Estrutura de objeto ELF

Fonte: Elaborada pelo autor (2023)

Um cabeçalho ELF reside no começo de um binário e guarda toda o mapa que descreve a organização do arquivo. A parte *Sections* guarda o conjunto de informações do objeto para a visualização do linking, como instruções, dados, tabela de símbolos, informação de realocação e outras. Já a tabela de cabeçalho do programa (*Program Header Table*) fala para o sistema como criar a imagem do processo. Arquivos usados para construir uma imagem de processo precisam ter uma tabela de cabeçalho do programa; arquivos realocáveis não precisam de um. Uma tabela de seções contém informações descrevendo as diferentes seções do arquivo. Cada uma das seções tem uma entrada na tabela, cada entrada fornece informações como o nome da seção, seu tamanho entre outras. Arquivos usados durante a etapa de linking precisam ter uma tabela de seção.

4.1.1 Representação de dados

O formato de arquivo objeto suporta vários processadores com bytes de 8 bits e arquiteturas de 32 bits. No entanto, foi projetado para ser expansível para arquiteturas maiores (ou menores). Os arquivos objeto, portanto, representam alguns dados de controle com um formato independente da máquina, tornando possível identificar os arquivos objeto e interpretar seu conteúdo de maneira comum. Os dados restantes em um arquivo objeto utilizam a codificação do processador de destino, independentemente da máquina na qual o arquivo foi criado.

Todas as estruturas de dados definidas pelo formato de arquivo objeto seguem as diretrizes de tamanho e alinhamento "naturais" para a classe relevante. Se necessário, as estruturas de dados contêm preenchimento explícito para garantir o

alinhamento de 4 bytes para objetos de 4 bytes, forçar tamanhos de estruturas a serem múltiplos de 4, etc. Os dados também possuem alinhamento adequado a partir do início do arquivo. Portanto, por exemplo, uma estrutura contendo um membro `Elf32_Addr` será alinhada em um espaço de 4 bytes dentro do arquivo. Ou seja, os tipos de dados terão um tamanho e alinhamento padrão para que possam ser usados em qualquer arquitetura. A figura “Tipos de dados seu tamanho e alinhamento padrão” mostra alguns exemplos desse padrão.

Name	Size	Alignment	Purpose
<code>Elf32_Addr</code>	4	4	Unsigned program address
<code>Elf32_Half</code>	2	2	Unsigned medium integer
<code>Elf32_Off</code>	4	4	Unsigned file offset
<code>Elf32_Sword</code>	4	4	Signed large integer
<code>Elf32_Word</code>	4	4	Unsigned large integer
<code>unsigned char</code>	1	1	Unsigned small integer

Figura 2 – Tipos de dados seu tamanho e alinhamento padrão
Fonte: Elaborada pelo autor (2023)

4.1.2 Cabeçalho ELF

Algumas estruturas de controle de arquivo objeto podem crescer, pois o cabeçalho ELF contém seus tamanhos reais. Se o formato do arquivo objeto for alterado, um programa pode encontrar estruturas de controle maiores ou menores do que o esperado. Portanto, os programas podem ignorar informações "extras". O tratamento de informações "ausentes" depende do contexto e será especificado quando e se extensões forem definidas. A figura “Cabeçalho ELF” representa a estrutura base de um cabeçalho ELF.

```
#define EI_NIDENT      16

typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half      e_type;
    Elf32_Half      e_machine;
    Elf32_Word      e_version;
    Elf32_Addr      e_entry;
    Elf32_Off       e_phoff;
    Elf32_Off       e_shoff;
    Elf32_Word      e_flags;
    Elf32_Half      e_ehsize;
    Elf32_Half      e_phentsize;
    Elf32_Half      e_phnum;
    Elf32_Half      e_shentsize;
    Elf32_Half      e_shnum;
    Elf32_Half      e_shstrndx;
} Elf32_Ehdr;
```

Figura 3 – Cabeçalho ELF

Fonte: Elaborada pelo autor (2023)

Vamos a explicação de cada campo:

Campo	Descrição																								
e_ident	Este conjunto de bytes especifica como interpretar o arquivo, independentemente do processador ou dos conteúdos restantes do arquivo. Dentro deste conjunto, tudo é nomeado por meio de macros, que começam com o prefixo EI_ e podem conter valores que começam com o prefixo ELF.																								
e_type	Esse campo identifica o tipo do objeto <table><thead><tr><th>Nome</th><th>Valor</th><th>Significado</th></tr></thead><tbody><tr><td>ET_NONE</td><td>0</td><td>Sem tipo</td></tr><tr><td>ET_REL</td><td>1</td><td>Realocável</td></tr><tr><td>ET_EXEC</td><td>2</td><td>Executável</td></tr><tr><td>ET_DYN</td><td>3</td><td>Objeto compartilhado</td></tr><tr><td>ET_CORE</td><td>4</td><td>Arquivo Core</td></tr><tr><td>ET_LOPROC</td><td>0xff00</td><td>Específico do processador</td></tr><tr><td>ET_HIPROC</td><td>0xffff</td><td>Específico do processador</td></tr></tbody></table> Embora os conteúdos do arquivo core sejam indefinidos, o tipo ET_CORE é reservado para marcar o arquivo. Os valores de ET_LOPROC a ET_HIPROC (inclusive) são reservados para semânticas específicas do processador. Outros valores são reservados e serão atribuídos a novos tipos de arquivo objeto conforme necessário.	Nome	Valor	Significado	ET_NONE	0	Sem tipo	ET_REL	1	Realocável	ET_EXEC	2	Executável	ET_DYN	3	Objeto compartilhado	ET_CORE	4	Arquivo Core	ET_LOPROC	0xff00	Específico do processador	ET_HIPROC	0xffff	Específico do processador
Nome	Valor	Significado																							
ET_NONE	0	Sem tipo																							
ET_REL	1	Realocável																							
ET_EXEC	2	Executável																							
ET_DYN	3	Objeto compartilhado																							
ET_CORE	4	Arquivo Core																							
ET_LOPROC	0xff00	Específico do processador																							
ET_HIPROC	0xffff	Específico do processador																							
e_machine	Esse campo especifica a arquitetura necessária para que o programa execute.																								
e_version	Identifica a versão do arquivo.																								

e_entry	Esse campo fornece o endereço virtual para o qual o sistema primeiro transfira controle, consequentemente iniciando o processo.
e_phoff	Esse campo mantém o <i>offset</i> do cabeçalho do programa em bytes.
e_shoff	Contém o <i>offset</i> da tabela de seções.
e_flags	Contém <i>flags</i> associadas aos processos do processador.
e_ehsize	Armazena o tamanho do cabeçalho ELF em bytes.
e_phentsize	Tamanho em bytes de uma entrada dentro da tabela do cabeçalho do programa.
e_phnum	Identifica o número de campos na tabela de cabeçalho do programa em bytes.
e_shentsize	Fornece o tamanho da tabela de seções em bytes.
e_shnum	Número de campos da tabela de seções em bytes.
e_shstrndx	Contém o index da tabela de seções associados aos nomes das seções.

Tabela 1 – Campos do cabeçalho ELF

Fonte: Elaborada pelo autor (2023)

Vamos ver um exemplo abaixo para deixar mais clara a explicação:

* struct e_ident e_ident		0h	10h	Fg:	Magic number and other info
char file_identification[4]	"ELF"	0h	4h	Fg:	
enum ei_class_2_e ei_class_2	ELFCLASS64 (2)	4h	1h	Fg:	
enum ei_data_2_e ei_data_2	ELFDATA2LSB (1)	5h	1h	Fg:	
enum ei_version_2_e ei_version_2	E_CURRENT (1)	6h	1h	Fg:	
enum ei_osabi_2_e ei_osabi_2	ELFOSABI_NONE ...	7h	1h	Fg:	
uchar ei_abiversion	0	8h	1h	Fg:	
uchar ei_pad[6]		9h	6h	Fg:	
uchar ei_nident_SIZE	0	Fh	1h	Fg:	
enum e_type64_e e_type	ET_DYN (3)	10h	2h	Fg:	Object file type
enum e_machine64_e e_machine	EM_X86_64 (62)	12h	2h	Fg:	Architecture
enum e_version64_e e_version	EV_CURRENT (1)	14h	4h	Fg:	Object file version
Elf64_Addr e_entry_START_ADDRESS	0x000000000000...	18h	8h	Fg:	Entry point virtual address
Elf64_Off e_phoff_PROGRAM_HEADER_OFFSET_IN_FILE	64	20h	8h	Fg:	Program header table file offset
Elf64_Off e_shoff_SECTION_HEADER_OFFSET_IN_FILE	863848	28h	8h	Fg:	Section header table file offset
Elf32_Word e_flags	0	30h	4h	Fg:	Processor-specific flags
Elf64_Half e_ehsize_ELF_HEADER_SIZE	64	34h	2h	Fg:	ELF Header size in bytes
Elf64_Half e_phentsize_PROGRAM_HEADER_ENTRY_SIZE_IN_FILE	56	36h	2h	Fg:	Program header table entry size
Elf64_Half e_phnum_NUMBER_OF_PROGRAM_HEADER_ENTRIES	13	38h	2h	Fg:	Program header table entry count
Elf64_Half e_shentsize_SECTION_HEADER_ENTRY_SIZE	64	3Ah	2h	Fg:	Section header table entry size
Elf64_Half e_shnum_NUMBER_OF_SECTION_HEADER_ENTRIES	30	3Ch	2h	Fg:	Section header table entry count
Elf64_Half e_shtrndx_STRING_TABLE_INDEX	29	3Eh	2h	Fg:	Section header string table index

Figura 4 – Exemplo de Cabeçalho ELF

Fonte: Elaborada pelo autor (2023)

Conforme vimos, a primeira seção é a `e_ident` que identifica o magic number, ou identificador do arquivo, no nosso caso se trata de um arquivo ELF. Modificando esse campo podemos ocultar a assinatura do arquivo. A segunda seção `ei_class` fornece a classe do arquivo, e conforme nossa tabela podemos ver que ele é do tipo objeto compartilhado, uma vez que seu valor é `ET_DYN(3)`. A arquitetura para ele executar podemos ver no campo `e_machine64` que fornece o valor `x86_64`. `E_version` é current, e o endereço virtual de entrada do programa pode ser visto mais claramente abaixo, mas é mostrado em `e_entry_START_ADDRESS`:

Elf64_Addr e_entry_START_ADDRESS	0x0000000000017D10
----------------------------------	--------------------

Figura 5 – Entry Point

Fonte: Elaborada pelo autor (2023)

Próximo campo `e_phoff` fornece o offset da tabela do cabeçalho do programa que representa 64 bytes e o offset da tabela de seções (`e_shoff`) é 863848. Quanto as `e_flags` não possui nenhuma configurada como podemos ver no campo dele (0). O tamanho do cabeçalho do arquivo ELF (`e_hsize`) é 64 bytes, já o tamanho da tabela do cabeçalho do programa (`e_phentsize`) é de 56 bytes e possui 13 entradas (`e_phnum`).

Por sua vez a tabela de seção possui um tamanho de 64 bytes (`e_shentsize`) e 30 entradas (`e_shnum`). O último campo fornece para nós informações sobre o index da tabela de seção que equivale a 29 (`e_shtrndx`).

4.1.3 Cabeçalho do Programa (Phdr)

A tabela de cabeçalho de programa de um arquivo executável ou objeto compartilhado é um array de estruturas, cada uma descrevendo um segmento ou outras informações necessárias para o sistema preparar o programa para execução. Um segmento de arquivo objeto contém uma ou mais seções. Os cabeçalhos de programa são relevantes apenas para arquivos executáveis e objetos compartilhados. Um arquivo especifica o tamanho de seu próprio cabeçalho de programa por meio dos membros `e_phentsize` e `e_phnum` do cabeçalho ELF.

Uma tabela de cabeçalho de programa em um arquivo executável ou objeto compartilhado é uma coleção de estruturas que descrevem segmentos ou outras informações necessárias para o sistema preparar o programa para a execução. Um segmento em um arquivo objeto contém uma ou mais seções. Os cabeçalhos de programa são relevantes apenas para arquivos executáveis e objetos compartilhados. Um arquivo especifica o tamanho de sua própria tabela de cabeçalho de programa através dos membros `e_phentsize` e `e_phnum` do cabeçalho ELF.

É importante ressaltar que existe uma singela diferença entre a tabela do cabeçalho de programa de um arquivo ELF32 para um ELF64, que está na mudança de lugar do campo `p_flag`. Contudo tirando esse detalhe ambos seguem a mesma estrutura:

```
typedef struct {
    uint32_t    p_type;
    Elf32_Off   p_offset;
    Elf32_Addr  p_vaddr;
    Elf32_Addr  p_paddr;
    uint32_t    p_filesz;
    uint32_t    p_memsz;
    uint32_t    p_flags;
    uint32_t    p_align;
} Elf32_Phdr;

typedef struct {
    uint32_t    p_type;
    uint32_t    p_flags;
    Elf64_Off   p_offset;
    Elf64_Addr  p_vaddr;
    Elf64_Addr  p_paddr;
    uint64_t    p_filesz;
    uint64_t    p_memsz;
    uint64_t    p_align;
} Elf64_Phdr;
```

Figura 6 – Cabeçalho ELF32 e ELF64
Fonte: Elaborada pelo autor (2023)

Campo	Descrição
p_type	Indica qual o tipo de segmento que esse elemento descreve ou como interpretá-lo.
p_offset	Offset do início do arquivo até o primeiro byte no qual o segmento reside.
p_vaddr	Endereço virtual no qual o segmento reside.
p_paddr	Para sistemas nos quais o endereço físico é relevante esse campo mostra esse dado.
p_filesz	Guarda o número de bytes da imagem do arquivo no segmento.
p_memsz	Número de bytes na imagem de memória do segmento.
p_flags	Uma máscara relevante para o segmento, por exemplo caso ele possa ser executado, escrito ou lido (PF_X, PF_W, PF_R).
p_align	Contém o valor ao qual o segmento está alinhado na memória e no arquivo.

Tabela 2 – Cabeçalho do Programa
Fonte: Elaborada pelo autor (2023)

Continuando nosso exemplo com as informações acima podemos agora interpretar a primeira tabela de cabeçalho do programa (sim, existem mais de uma no mesmo binário para direcionar como a execução deve ser feita):

▼ struct program_table_entry64_t program_table_element[13]	
▼ struct program_table_entry64_t program_table_element[0]	(R_) Program Header
enum p_type64_e p_type	PT_PHDR (6)
enum p_flags64_e p_flags	PF_Read (4)
Elf64_Off p_offset_FROM_FILE_BEGIN	40h
Elf64_Addr p_vaddr_VIRTUAL_ADDRESS	0x0000000000000040
Elf64_Addr p_paddr_PHYSICAL_ADDRESS	0x0000000000000040
Elf64_Xword p_filesz_SEGMENT_FILE_LENGTH	728
Elf64_Xword p_memsz_SEGMENT_RAM_LENGTH	728
Elf64_Xword p_align	8

Figura 7 – Exemplo de tabela de cabeçalho de programa
Fonte: Elaborada pelo autor (2023)

Aqui teremos primeiro o p_type, o qual fornece o valor PHDR, o que representa o tamanho e localização da tabela do cabeçalho do programa, tanto no arquivo quanto na memória da imagem do programa. O p_flag possui um valor PF_READ (PF_R) dizendo que esse segmento deve ser lido apenas. O p_offset fornece o offset do segmento (40h - o que equivale a 40 em hexadecimal ou 64 em decimal). E o primeiro byte que contém um pedaço do segmento (p_vaddr) é o 0x0000000000000040, que para esse caso também é igual ao endereço físico (p_paddr). O tamanho do segmento

é 728 bytes (`p_filesz`) e ocupam 728 bytes na memória (`p_memsz`). Seu alinhamento em relação a memória e arquivo é de 8 bytes.

4.1.4 Tabela de Seções (Shdr)

A tabela de cabeçalho de seção de um arquivo objeto permite localizar todas as seções do arquivo. A tabela de cabeçalho de seção é um array de estruturas `Elf32_Shdr`, conforme descrito abaixo. Um índice da tabela de cabeçalho de seção é um subscrito nesse array. O membro `e_shoff` do cabeçalho ELF fornece o deslocamento em bytes a partir do início do arquivo até a tabela de cabeçalho de seção; `e_shnum` indica quantas entradas a tabela de cabeçalho de seção contém; e `e_shentsize` fornece o tamanho em bytes de cada entrada.

Alguns índices da tabela de cabeçalho de seção são reservados; um arquivo objeto não terá seções para esses índices especiais.

Os primeiros índices da tabela de seções não possuem campos específicos para eles, e são chamados de índices especiais:

SHN_UNDEF: Informa que a seção está indefinida, ausente, é irrelevante ou desprezível.

SHN_LORESERVE: Fornece o limite inferior do intervalo de índices reservados.

SHN_LOPROC, SHN_HIPROC: Valores usados por operações específicas do processador.

SHN_ABS: Especifica o valor absoluto para a referência correspondente. Por exemplo, um símbolo definido relativamente ao número da seção `SHN_ABS` tem um valor absoluto e não é afetado por realocação.

SHN_COMMON: Símbolos definidos relativos a esta seção são símbolos comuns, como o `COMMON` da linguagem FORTRAN ou `unallocated` da linguagem C.

SHN_HIRESERVE: Esse valor especifica o limite superior do intervalo reservado para os índices.

Quanto aos campos relativos a tabela de seção, podemos vê-lo abaixo na figura “Tabela de seção”.

```

typedef struct {
    uint32_t sh_name;
    uint32_t sh_type;
    uint32_t sh_flags;
    Elf32_Addr sh_addr;
    Elf32_Off sh_offset;
    uint32_t sh_size;
    uint32_t sh_link;
    uint32_t sh_info;
    uint32_t sh_addralign;
    uint32_t sh_entsize;
} Elf32_Shdr;

typedef struct {
    uint32_t sh_name;
    uint32_t sh_type;
    uint64_t sh_flags;
    Elf64_Addr sh_addr;
    Elf64_Off sh_offset;
    uint64_t sh_size;
    uint32_t sh_link;
    uint32_t sh_info;
    uint64_t sh_addralign;
    uint64_t sh_entsize;
} Elf64_Shdr;

```

Figura 8 – Tabela de seção
Fonte: Elaborada pelo autor (2023)

Campo	Descrição
sh_name	Especifica o nome da seção.
sh_type	Categoriza a seção conforme seu conteúdo.
sh_flags	Descreve atributos da seção como se ela deve ser escrita durante a execução, ou se a seção ocupa um espaço de memória.
sh_addr	Endereço no qual o primeiro byte da seção deve estar.
sh_offset	Offset do início do arquivo no qual o primeiro byte da seção deve residir.
sh_size	Tamanho da seção em bytes.
sh_link	Mantém um link da tabela do cabeçalho de seção.
sh_info	Informações extras.
sh_addralign	Alinhamento por conta de restrições de seções especiais.
sh_entsize	Algumas seções guardam uma tabela de valores de tamanho fixo, como uma tabela de símbolo e aqui são fornecidos os detalhes do tamanho da seção.

Tabela 3 – Campos da tabela de seção
Fonte: Elaborada pelo autor (2023)

Várias seções guardam informações do programa e de controle cujos campos acima ajudam a especificar. Vamos ver algumas delas abaixo:

- **A seção .bss** armazena dados não inicializados que contribuem para a imagem de memória do programa. Por definição, o sistema inicializa

esses dados com zeros quando o programa começa a ser executado. Essa seção é do tipo SHT_NOBITS. Os atributos são SHF_ALLOC e SHF_WRITE.

- **.comment** contém informações de controle de versão. Essa seção é do tipo SHT_PROGBITS. Não são usados atributos.
- **.ctors** armazena ponteiros inicializados para as funções construtoras de C++. Essa seção é do tipo SHT_PROGBITS. Os atributos são SHF_ALLOC e SHF_WRITE.
- **.data** armazena dados inicializados que contribuem para a imagem de memória do programa. Essa seção é do tipo SHT_PROGBITS. Os atributos são SHF_ALLOC e SHF_WRITE.
- **.data1** armazena dados inicializados que contribuem para a imagem de memória do programa. Essa seção é do tipo SHT_PROGBITS. Os atributos são SHF_ALLOC e SHF_WRITE.
- **.debug** armazena informações para depuração simbólica. O conteúdo é indefinido. Essa seção é do tipo SHT_PROGBITS. Não são usados atributos.
- **.dtors** armazena ponteiros inicializados para as funções destrutoras de C++. Essa seção é do tipo SHT_PROGBITS. Os atributos são SHF_ALLOC e SHF_WRITE.
- **.dynamic** armazena informações para ligação dinâmica. Os atributos da seção incluirão o bit SHF_ALLOC. Se o bit SHF_WRITE estiver definido, isso depende do processador específico. Essa seção é do tipo SHT_DYNAMIC. Consulte os atributos acima.
- **.dynstr** armazena strings (cadeia de caracteres) necessárias para ligação dinâmica, comumente as strings que representam os nomes associados às entradas da tabela de símbolos. Essa seção é do tipo SHT_STRTAB. O atributo usado é SHF_ALLOC.
- **.dynsym** armazena a tabela de símbolos para ligação dinâmica. Essa seção é do tipo SHT_DYNSYM. O atributo usado é SHF_ALLOC.
- **.fini** armazena instruções executáveis que contribuem para o código de terminação do processo. Quando um programa termina normalmente, o sistema executa o código dessa seção. Essa seção é do tipo

SHT_PROGBITS. Os atributos usados são SHF_ALLOC e SHF_EXECINSTR.

- **.gnu.version** armazena a tabela de símbolos de versão, um array de elementos `ElfN_Half`. Essa seção é do tipo `SHT_GNU_versym`. O atributo usado é `SHF_ALLOC`.
- **.gnu.version_d** armazena as definições dos símbolos de versão, uma tabela de estruturas `ElfN_Verdef`. Essa seção é do tipo `SHT_GNU_verdef`. O atributo usado é `SHF_ALLOC`.
- **.gnu.version_r** armazena os elementos necessários dos símbolos de versão, uma tabela de estruturas `ElfN_Verneed`. Essa seção é do tipo `SHT_GNU_versym`. O atributo usado é `SHF_ALLOC`.
- **.got** armazena a tabela de deslocamento global. Essa seção é do tipo `SHT_PROGBITS`. Os atributos são específicos do processador.
- **.hash** armazena uma tabela hash de símbolos. Essa seção é do tipo `SHT_HASH`. O atributo usado é `SHF_ALLOC`.
- **.init** armazena instruções executáveis que contribuem para o código de inicialização do processo. Quando um programa começa a ser executado, o sistema executa o código dessa seção antes de chamar o ponto de entrada do programa principal. Essa seção é do tipo `SHT_PROGBITS`. Os atributos usados são `SHF_ALLOC` e `SHF_EXECINSTR`.
- **.interp** armazena o caminho do interpretador de programa. Se o arquivo tiver um segmento carregável que inclua a seção, os atributos da seção incluirão o bit `SHF_ALLOC`. Caso contrário, esse bit estará desligado. Essa seção é do tipo `SHT_PROGBITS`.
- **.line** armazena informações de números de linha para depuração simbólica, descrevendo a correspondência entre o código-fonte do programa e o código de máquina. O conteúdo é indefinido. Essa seção é do tipo `SHT_PROGBITS`. Não são usados atributos.
- **.note** armazena várias notas. Essa seção é do tipo `SHT_NOTE`. Não são usados atributos.
- **.note.ABI-tag** é usada para declarar a ABI esperada em tempo de execução da imagem ELF. Ela pode incluir o nome do sistema operacional

e suas versões em tempo de execução. Essa seção é do tipo SHT_NOTE. O único atributo usado é SHF_ALLOC.

- **.note.gnu.build-id** é usada para armazenar um ID que identifica de forma única o conteúdo da imagem ELF. Arquivos diferentes com o mesmo ID de build devem conter o mesmo conteúdo executável. Consulte a opção `--build-id` do linker(ld) para obter mais detalhes. Essa seção é do tipo SHT_NOTE. O único atributo usado é SHF_ALLOC.
- **.note.GNU-stack** é usada em arquivos de objeto do Linux para declarar atributos de pilha. Essa seção é do tipo SHT_PROGBITS. O único atributo usado é SHF_EXECINSTR. Isso indica ao linker GNU que o arquivo de objeto requer uma pilha executável.
- **.note.openbsd.ident** é comumente encontrada em executáveis nativos do OpenBSD e serve para identificar o arquivo, permitindo que o kernel ignore testes de emulação binária ELF de compatibilidade ao carregar o arquivo.
- **.plt** armazena a tabela de procedimentos de ligação (procedure linkage table). Essa seção é do tipo SHT_PROGBITS. Os atributos são específicos do processador.
- **.relNAME** e **.relaNAME** armazenam informações de realocação, como descrito abaixo. Se o arquivo tiver um segmento carregável que inclua realocação, os atributos da seção incluirão o bit SHF_ALLOC. Caso contrário, esse bit estará desligado. Por convenção, "NAME" é fornecido pela seção à qual as realocações se aplicam. Assim, uma seção de realocação para `.text` normalmente teria o nome `.rel.text` (no caso de `.relNAME`) ou `.rela.text` (no caso de `.relaNAME`). Essas seções são do tipo SHT_REL (para `.relNAME`) e SHT_RELA (para `.relaNAME`).
- **.rodata** e **.rodata1** armazenam dados somente leitura que normalmente contribuem para um segmento não gravável na imagem do processo. Essas seções são do tipo SHT_PROGBITS. O atributo usado é SHF_ALLOC.
- **.shstrtab** armazena os nomes das seções. Essa seção é do tipo SHT_STRTAB. Não são usados atributos.

- **.strtab** armazena strings, mais comumente as strings que representam os nomes associados às entradas da tabela de símbolos. Se o arquivo tiver um segmento carregável que inclua a tabela de strings de símbolos, os atributos da seção incluirão o bit SHF_ALLOC. Caso contrário, esse bit estará desligado. Essa seção é do tipo SHT_STRTAB.
- **.symtab** armazena uma tabela de símbolos. Se o arquivo tiver um segmento carregável que inclua a tabela de símbolos, os atributos da seção incluirão o bit SHF_ALLOC. Caso contrário, esse bit estará desligado. Essa seção é do tipo SHT_SYMTAB.
- **.text** armazena o “text”, isto é, as instruções executáveis de um programa. Essa seção é do tipo SHT_PROGBITS. Os atributos usados são SHF_ALLOC e SHF_EXECINSTR.

4.1.5 Tabela de String e Símbolos

As seções de tabela de strings armazenam sequências de caracteres terminadas por null, comumente chamadas de strings. O arquivo objeto utiliza essas strings para representar nomes de símbolos e seções. Uma string é referenciada como um índice na seção de tabela de strings.

O primeiro byte, que corresponde ao índice zero, é definido para armazenar um byte nulo (“\0”). Da mesma forma, o último byte da tabela de strings é definido para armazenar um byte nulo, garantindo a terminação nula de todas as strings.

A tabela de símbolos de um arquivo objeto contém informações necessárias para localizar e realocar as definições e referências simbólicas de um programa. Um índice na tabela de símbolos é um subscrito nesse array.

Index	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9
0	\0	n	a	m	e	.	\0	v	a	r
10	i	a	b	l	e	\0	a	b	l	e
20	\0	\0	x	x	\0					

Figura 9 – Tabela de símbolos e strings
Fonte: Elaborada pelo autor (2023)

Index	String
0	<i>none</i>
1	name.
7	Variable
11	able
16	able
24	<i>null string</i>

Figura 10 – Tabela de índice e strings
Fonte: Elaborada pelo autor (2023)

4.1.6 Tabela de realocação (Rel e Rela)

A realocação é o processo de vincular referências simbólicas a definições simbólicas. Arquivos relocáveis devem conter informações que descrevam como modificar o conteúdo de suas seções, permitindo que arquivos executáveis e objetos compartilhados armazenem as informações corretas para a imagem do programa de um processo. As entradas de realocação são esses dados.

4.1.7 Dynamic tags (Dyn)

A seção `.dynamic` section contém uma série de estrutura que contém informações relevantes para o processo de linking dinâmico (Dynamic linking) que veremos em uma seção mais à frente.

Por hora basta saber que Dynamic linking é o mecanismo por meio do qual múltiplos programas conseguem compartilhar e usar bibliotecas ou recursos de código durante o tempo de execução.

4.1.8 Notes (Nhdr)

As notas ELF permitem a adição de informações arbitrárias para que o sistema possa utilizá-las. Elas são amplamente utilizadas em arquivos core (com `e_type` definido como `ET_CORE`), mas muitos projetos definem seu próprio conjunto de extensões. Por exemplo, a cadeia de ferramentas GNU utiliza notas ELF para transmitir informações do linker para a biblioteca C.

As seções de notas contêm uma série de notas (consulte as definições de estrutura abaixo). Cada nota é seguida pelo campo de nome (cujo tamanho é definido em `n_namesz`) e, em seguida, pelo campo de descritor (cujo tamanho é definido em `n_descsz`) e cujo endereço de início possui um alinhamento de 4 bytes. Nenhum desses campos é definido na estrutura de nota devido aos seus comprimentos arbitrários.

4.2 Formato COFF

O COFF significa Common Object File Format. É um formato de arquivo usado para armazenar código compilado, como o produzido por um compilador ou um linker.

Assim como a maioria dos formatos de arquivo de compilador, o COFF define estruturas dentro do arquivo para armazenar informações sobre as seções do programa, como `.text` e `.data`, e sobre os símbolos que o programa declara ou define.

O COFF pode ser usado para armazenar funções individuais ou símbolos, fragmentos de programas, bibliotecas ou executáveis completos.

O formato executável Microsoft PE (strictly PE/COFF) contém uma versão do COFF.

O COFF é uma implementação de um formato de arquivo objeto com o mesmo nome que foi desenvolvido pela AT&T para ser utilizado em sistemas baseados em UNIX. Esse formato promove a programação modular e oferece métodos poderosos e flexíveis para gerenciar segmentos de código e a memória do sistema-alvo.

4.2.1 Estrutura do COFF

Os elementos de um arquivo de objeto COFF descrevem as seções do arquivo e as informações para depuração simbólica. Estes elementos incluem:

- Um cabeçalho do arquivo.
- Informações de cabeçalho opcionais.
- Uma tabela de cabeçalhos de seção.
- Dados brutos para cada seção inicializada.
- Informações de realocação para cada seção inicializada.
- Uma tabela de símbolos.

- Uma tabela de strings.

O montador e a etapa de ligação produzem arquivos de objeto com a mesma estrutura COFF; no entanto, um programa que é ligado pela última vez geralmente não contém entradas de realocação. A figura abaixo ilustra a estrutura do arquivo de objeto.

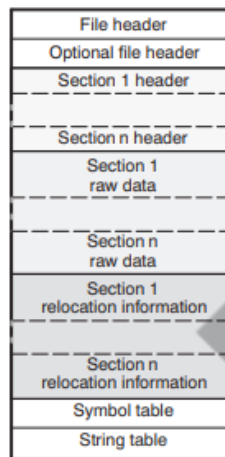


Figura 11 – Estrutura arquivo COFF
Fonte: TEXAS INSTRUMENT (2009)

4.2.2 File Header

Com 22 bytes de informações, o cabeçalho do arquivo descreve o formato geral de um arquivo de objeto. A estrutura do cabeçalho do arquivo COFF é demonstrada na tabela “Campos do file header do COFF”.

Byte	Tipo	Descrição
0-1	Unsigned short	ID da versão; indica a versão da estrutura do arquivo COFF.
2-3	Unsigned short	Número de cabeçalhos de seção.
4-7	Integer	Carimbo de data e hora; indica quando o arquivo foi criado.
8-11	Integer	Ponteiro de arquivo; contém o endereço de início da tabela de símbolos.
12-15	Integer	Número de entradas na tabela de símbolos.
16-17	Unsigned short	Número de bytes no cabeçalho opcional. Este campo é 0 ou 28; se for 0, não existe cabeçalho de arquivo opcional.
18-19	Unsigned short	Flags (veja Tabela 2).
20-21	Unsigned short	ID do alvo; magic number (veja Tabela 3) indica que o arquivo pode ser executado em um sistema TI específico.

Tabela 4 – Campos do file header do COFF
Fonte: Elaborada pelo autor (2023)

A tabela “*Flags do file header do COFF*” fornece uma lista das flags que podem aparecer nos bytes de 18 e 19 do file header.

Mnemônico	Flag	Descrição
F_RELFLG	0001h	As informações de realocação foram removidas do arquivo.
F_EXEC	0002h	O arquivo é realocável (não contém referências externas não resolvidas).
F_LNNO(1)	0004h	Números de linha foram removidos do arquivo. Para outros alvos: Reservado.
F_LSYMS	0008h	Símbolos locais foram removidos do arquivo.
F_LITTLE	0100h	O alvo é um dispositivo little-endian.
F_BIG(1)	0200h	O alvo é um dispositivo big-endian. Para outros alvos: Reservado.
F_SYMMERGE(1)	1000h	Símbolos duplicados foram removidos.

Tabela 5 – Flags do file header do COFF
Fonte: Elaborada pelo autor (2023)

4.2.3 Estrutura da tabela de seção

Arquivos de objeto COFF incluem uma tabela com cabeçalhos de seção, que determinam o início de cada seção no arquivo. Cada seção é caracterizada pelo seu próprio cabeçalho. A tabela “Campos tabela de seção” demonstra a estrutura de cada um desses cabeçalhos de seção.

Número do Byte	Tipo	Descrição
0-7	Caractere	Este campo contém um dos seguintes: 1) Um nome de seção de 8 caracteres preenchido com nulos. 2) Um ponteiro para a tabela de strings se o nome do símbolo for maior que oito caracteres.
8-11	Long(1)	Endereço físico da seção.
12-15	Long(1)	Endereço virtual da seção.
16-19	Long(1)	Tamanho da seção em bytes ou palavras.
20-23	Long(1)	Ponteiro de arquivo para dados brutos.
24-27	Long(1)	Ponteiro de arquivo para entradas de realocação.
28-31	Long(1)	Reservado.
32-35	Unsigned long(2)	Número de entradas de realocação.
36-39	Unsigned long(2)	Número de entradas de número de linha. Para outros dispositivos: Reservado.
40-43	Unsigned long(2)	Flags (veja Tabela 6).
44-45	Unsigned short	Reservado.
46-47	Unsigned short	Número de página de memória.

Tabela 6 – Campos tabela de seção
Fonte: Elaborada pelo autor (2023)

A tabela “Flags para tabela de seção” fornece as flags que podem ser usadas dentro do campo 40 a 43:

Mnemônico	Flag	Descrição
STYP_REG	00000000h	Seção regular (alocada, realocada, carregada).
STYP_DSECT	00000001h	Seção fictícia (realocada, não alocada, não carregada).
STYP_NOLOAD	00000002h	Seção sem carga (alocada, realocada, não carregada).
STYP_GROUP(2)	00000004h	Seção agrupada (formada a partir de várias seções de entrada). Outros dispositivos: Reservado.
STYP_PAD(2)	00000008h	Seção de preenchimento (carregada, não alocada, não realocada). Outros dispositivos: Reservado.
STYP_COPY	00000010h	Seção de cópia (realocada, carregada, mas não alocada; entradas de realocação são processadas normalmente).
STYP_TEXT	00000020h	Seção contém código executável.
STYP_DATA	00000040h	Seção contém dados inicializados.
STYP_BSS	00000080h	Seção contém dados não inicializados.
STYP_BLOCK(3)	00001000h	Alinhamento usado como fator de bloqueio.
STYP_PASS(3)	00002000h	A seção deve passar sem alterações.
STYP_CLINK	00004000h	Seção requer ligação condicional.
STYP_VECTOR(4)	00008000h	Seção contém tabela de vetor.
STYP_PADDED(4)	00010000h	A seção foi preenchida.

Tabela 7 – Flags para tabela de seção
Fonte: Elaborada pelo autor (2023)

4.2.4 Seção de realocação

Cada referência realocável em um arquivo de objeto COFF possui uma entrada de realocação correspondente. Essas entradas são geradas automaticamente pelo compilador. Durante a etapa de link, tais entradas são lidas simultaneamente com cada seção de entrada, sendo então realizada a realocação. As entradas de realocação estabelecem o tratamento das referências dentro de cada seção de entrada.

Para todos os parâmetros usados para realizar realocação podemos conferir na documentação: <https://www.ti.com/lit/na/spraa08/spraa08.pdf?ts=1688953493796>.

Número do Byte	Tipo	Descrição
0-3	Long	Endereço virtual da referência.
4-5	Short	Índice da tabela de símbolos (0-65 535).
6-7	Unsigned short	Reservado.
8-9	Unsigned short	Tipo de realocação.

Tabela 8 – Realocação
Fonte: Elaborada pelo autor (2023)

Vamos a um exemplo de realocação para melhor entender o processo.

O endereço virtual corresponde ao endereço do símbolo na seção corrente antes da realocação, indicando onde a realocação deve acontecer. (Esse é o endereço do campo no código do objeto que necessita ser corrigido).

```

2          .global X
3          00000000 !00000012 b X

```

Neste caso, o endereço virtual do campo realocável é 0001.

O índice na tabela de símbolos representa a posição do símbolo referenciado. No exemplo acima, esse campo contém o índice de X na tabela de símbolos. O valor da realocação é a diferença entre a posição atual do símbolo na seção e sua posição no momento da montagem. O campo realocável precisa ser realocado pela mesma quantidade que o símbolo referenciado. No exemplo, X tem valor 0 antes da realocação. Imagine que X seja realocado para o endereço 2000h.

Este é o valor da realocação ($2000h - 0 = 2000h$), então o campo de realocação no endereço 1 é ajustado, adicionando-se 2000h a ele.

Se você souber em qual seção um símbolo está definido, poderá determinar o endereço realocado desse símbolo. Por exemplo, se X está definido em .data e .data é realocado por 2000h, X também é realocado por 2000h.

Se o índice na tabela de símbolos em uma entrada de realocação é -1 (0FFFFh), isso é conhecido como realocação interna. Neste caso, o valor da realocação é simplesmente a quantidade pela qual a seção atual está sendo realocada.

O tipo de realocação determina o tamanho do campo que precisa ser corrigido e descreve como o valor corrigido é calculado. O campo de tipo depende do modo de endereçamento utilizado para gerar a referência realocável. No exemplo anterior para C6000, o endereço real do símbolo referenciado X é inserido em um campo de 8 bits no código do objeto. Este é um endereço de 8 bits, portanto, o tipo de realocação é R_RELBYTE.

4.2.5 Estrutura da tabela de símbolos e conteúdo

A estrutura da tabela de símbolos dos arquivos COFF pode ser vista na figura “Estrutura da tabela de símbolos do COFF”.

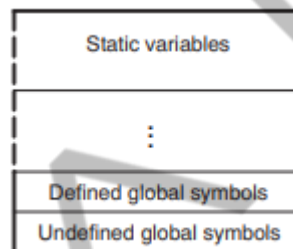


Figura 12 – Estrutura da tabela de símbolos do COFF
Fonte: TEXAS INSTRUMENT (2009)

As variáveis estáticas referem-se a símbolos definidos em C/C++ que possuem a classe de armazenamento estático fora de qualquer função. Se você tiver vários módulos que usam símbolos com o mesmo nome, torna-os estáticos restringe o escopo de cada símbolo ao módulo que o define (isso elimina conflitos de múltiplas definições). A entrada para cada símbolo na tabela de símbolos contém:

- Nome (ou um deslocamento para a tabela de strings).
- Tipo.
- Valor.
- Seção em que foi definido.
- Classe de armazenamento.

Todas as entradas de símbolos, independentemente da classe e tipo, possuem o mesmo formato na tabela de símbolos. Cada entrada da tabela de símbolos contém 18 bytes de informações listadas na tabela “Realocação”. Cada símbolo também pode ter uma entrada auxiliar de 18 bytes; os símbolos especiais listados na tabela “Campos da tabela de símbolos” sempre têm uma entrada auxiliar. Alguns símbolos podem não

ter todas as características mencionadas acima, se um campo específico não estiver definido, ele é definido como nulo.

Número do Byte	Tipo	Descrição
0-7	Char	Este campo contém um dos seguintes: 1) Um nome de símbolo de 8 caracteres, preenchido com nulos. 2) Um ponteiro para a tabela de strings, se o nome do símbolo for maior que oito caracteres.
8-11	Long(1)	Valor do símbolo; depende da classe de armazenamento.
12-13	Short	Número da seção do símbolo.
14-15	Unsigned short	Reservado.
16	Char	Classe de armazenamento do símbolo.
17	Char	Número de entradas auxiliares (sempre 0 ou 1).

Tabela 9 – Campos da tabela de símbolos
Fonte: Elaborada pelo autor (2023)

Símbolo	Descrição
.text	Endereço da seção .text
.data	Endereço da seção .data
.bss	Endereço da seção .bss
etext	Próximo endereço disponível após o final da seção de saída .text
edata	Próximo endereço disponível após o final da seção de saída .data
end	Próximo endereço disponível após o final da seção de saída .bss

Tabela 10 – Campos do cabeçalho de seção
Fonte: Elaborada pelo autor (2023)

4.2.6 Tabelas de strings

A tabela de strings armazena símbolos com nomes mais longos que oito caracteres. O campo na entrada da tabela de símbolos que normalmente conteria o nome do símbolo na verdade aponta para o nome do símbolo na tabela de strings. A tabela de strings armazena nomes de forma contígua, delimitados por um byte nulo. Os primeiros quatro bytes da tabela contêm o tamanho total da tabela em bytes; portanto, os deslocamentos para a tabela de strings são maiores ou iguais a 4.

5 COMPATIBILIDADE DE BINÁRIOS

5.1 Definição e importância da compatibilidade de binários

A compatibilidade binária se refere ao conceito em sistemas de computadores que permite que programas compilados sejam executados corretamente em versões subsequentes do mesmo sistema operacional ou em diferentes sistemas operacionais usando a mesma arquitetura de conjunto de instruções (ISA). É um aspecto importante do design do sistema, especialmente para software que se espera que seja portátil, reutilizável ou duradouro.

Em geral, um sistema compatível com binário fornece a garantia de que o software compilado para ser executado em uma versão de um sistema também pode ser executado em versões futuras do sistema, sem a necessidade de ser recompilado. Isso é crucial para a estabilidade e robustez dos ecossistemas de software, permitindo que os desenvolvedores distribuam binários de software pré-compilados que podem funcionar corretamente em diferentes sistemas de usuários.

Quando dois sistemas são compatíveis em binário, significa que programas criados para o primeiro sistema (usando um formato binário específico) serão executados no segundo sem quaisquer modificações. A compatibilidade binária cobre áreas como interfaces de chamada de sistema, interfaces de biblioteca e consistência de ABI (Interface Binária de Aplicativo).

5.2 Compatibilidade ABI

A interface binária de aplicação - ABI (do inglês *Application Binary Interface*) é uma especificação que define, entre outras coisas, como as funções são chamadas e como os dados são representados em formato binário. Se a ABI mudar de uma maneira incompatível, a compatibilidade binária será quebrada. Por exemplo, quando um programa chama uma função em uma biblioteca, a ABI define como essa chamada deve ser feita e como os resultados são retornados ao chamador.

A compatibilidade binária é crucial para a estabilidade e robustez de um sistema de computador. Em termos simples, a compatibilidade binária significa que um

programa compilado pode ser executado corretamente em diferentes versões do mesmo sistema operacional, ou em sistemas operacionais diferentes que usem a mesma arquitetura de conjunto de instruções (ISA).

Manter a compatibilidade binária muitas vezes requer que a ABI permaneça estável. Se uma ABI muda de uma maneira que não é retrocompatível, programas compilados anteriormente podem não funcionar corretamente ou podem não funcionar de todo. Portanto, ao desenvolver novas versões de um sistema operacional ou bibliotecas de software, é fundamental levar em conta a compatibilidade da ABI.

No nível mais básico, a ABI funciona como um protocolo de comunicação entre dois programas binários, como um programa de aplicativo e a biblioteca do sistema operacional.

Vamos explorar alguns dos elementos principais de uma ABI e como eles funcionam:

- **Chamadas de Função e Convenções de Registro:** Uma ABI define como as chamadas de função são realizadas em um nível binário. Por exemplo, ela especifica a ordem na qual os argumentos são empilhados e como a função é chamada e retorna um valor. Isso também incluem convenções de registro que determinam quais registradores do processador são usados para quais finalidades, quais registradores são preservados através de chamadas de função e assim por diante.
- **Layout de Memória e Alinhamento:** A ABI especifica o layout de dados na memória. Isso inclui o formato de dados complexos, como estruturas e uniões, além de regras de alinhamento de memória para diferentes tipos de dados. O alinhamento de memória se refere à maneira como os dados são organizados na memória, e diferentes arquiteturas de processador têm diferentes requisitos de alinhamento.
- **Formato de Arquivo Binário:** A ABI também define o formato dos arquivos binários que são executados. No caso do Linux, por exemplo, o formato de arquivo binário padrão é o ELF (*Executable and Linkable Format*). Este formato define coisas como a forma como o código e os dados são organizados no arquivo, como as dependências da biblioteca são resolvidas, e assim por diante.

- **ABI do Sistema:** AABI do sistema é a interface para o sistema operacional. Isso inclui as chamadas do sistema, os números das chamadas do sistema, e a forma como os sinais e as interrupções são tratados.

5.3 Interface do *system call*

A interface do *system call*, ou interface de chamada de sistema, é uma característica crítica na arquitetura de qualquer sistema operacional. Ela atua como um portal entre o espaço do usuário (no qual os programas de aplicação são executados) e o espaço do kernel (onde o sistema operacional executa suas operações mais privilegiadas e críticas).

As chamadas de sistema são a forma pela qual um programa no espaço do usuário solicita um serviço do sistema operacional. Isso pode incluir operações como criar ou remover arquivos, ler e escrever em um arquivo, alocar memória, criar processos e threads, e muitas outras ações. Essas chamadas são essenciais porque fornecem uma forma segura e controlada de acessar os recursos do sistema, mantendo a abstração entre o espaço do usuário e o espaço do kernel.

5.3.1 Como as chamadas de sistema funcionam

Quando um programa faz uma chamada de sistema, ele inicia uma solicitação para o kernel do sistema operacional. Esta solicitação é processada por meio de uma interrupção de software, que é uma forma de sinalizar ao processador para suspender o que está fazendo e atender a solicitação.

Ao receber esta interrupção, o processador muda do modo de usuário para o modo de kernel. Em seguida, ele passa o controle para um manipulador de interrupções no kernel, que é responsável por determinar qual chamada de sistema foi feita e como processá-la.

O kernel executa a operação solicitada e, em seguida, retorna o controle ao programa de usuário, juntamente com quaisquer resultados da chamada de sistema.

O conjunto específico de chamadas de sistema e como elas são acessadas é definido pela interface de chamada de sistema. Essa interface é um aspecto crítico da

interface binária de aplicação (ABI) do sistema operacional e é essencial para manter a compatibilidade binária.

A compatibilidade binária, neste contexto, significa que os programas compilados para fazer chamadas de sistema em uma versão particular do sistema operacional continuarão a funcionar corretamente mesmo quando executados em versões futuras do sistema operacional. Isso permite que os desenvolvedores de software criem programas que serão executados corretamente em uma ampla gama de versões do sistema operacional.

Manter a compatibilidade na interface de chamada de sistema pode ser um desafio. Os desenvolvedores do sistema operacional podem querer adicionar novas funcionalidades ou melhorar as existentes, o que pode exigir a adição de novas chamadas de sistema ou a alteração das existentes. No entanto, essas alterações devem ser feitas de forma que não quebrem a compatibilidade binária. Isso significa que as chamadas de sistema existentes não podem ser removidas ou alteradas de forma a modificar seu comportamento de uma maneira que afetaria negativamente os programas existentes.

Para resolver esse desafio, os desenvolvedores do sistema operacional geralmente optam por adicionar novas chamadas de sistema para fornecer novas funcionalidades, enquanto mantêm as chamadas de sistema existentes para garantir a compatibilidade com os programas existentes.

5.4 Compatibilidade de bibliotecas

A compatibilidade de bibliotecas é um conceito central na arquitetura de computadores, que se refere à capacidade de um programa para usar versões diferentes de uma biblioteca de software sem necessidade de modificação ou recompilação.

Este é um princípio importante na criação de software durável e escalável, permitindo que os programas existentes se beneficiem de melhorias e atualizações em bibliotecas de software sem precisar serem modificados ou recompilados.

As bibliotecas de software são coleções de código reutilizável que fornecem funcionalidades comumente necessárias por muitos programas diferentes. Elas podem

incluir tudo, desde rotinas matemáticas básicas, manipulação de strings e funções de entrada/saída até recursos mais complexos, como interfaces de rede, gráficos e interação com banco de dados.

A compatibilidade de bibliotecas pode ser dividida em dois aspectos principais: compatibilidade binária e compatibilidade de código-fonte.

5.4.1 Compatibilidade binária de biblioteca

A compatibilidade binária de bibliotecas é um aspecto fundamental para a interoperabilidade e reutilização do software. Ela permite que um programa compilado continue a funcionar com diferentes versões de uma biblioteca sem a necessidade de recompilação.

Quando compilamos um programa que faz uso de uma biblioteca, o compilador e o vinculador trabalham juntos para criar o executável final. Esse processo envolve a vinculação dos pontos de entrada da biblioteca (como funções ou variáveis) que o programa usa às suas respectivas localizações na biblioteca. Na prática, isso significa que o programa sabe exatamente onde procurar no arquivo da biblioteca para encontrar as funções e variáveis que está chamando.

O programa compilado depende não apenas da presença dessas funções e variáveis na biblioteca, mas também do layout exato e da assinatura dessas funções, que são definidos pela interface binária de aplicação (ABI). A ABI especifica detalhes como o layout de dados na memória, a organização de registros e o conjunto de instruções de máquina. Por exemplo, uma alteração na ABI poderia envolver a mudança de uma função para receber um argumento adicional ou a reorganização da estrutura de dados.

Se a ABI de uma biblioteca muda de uma versão para outra, pode haver problemas de compatibilidade binária. Por exemplo, se uma função for removida de uma biblioteca ou tiver sua assinatura alterada, um programa que foi compilado para usar essa função na biblioteca antiga não funcionará corretamente com a nova versão da biblioteca. O programa pode tentar chamar uma função que não existe mais, ou passar os argumentos errados para uma função, resultando em comportamento indefinido.

Manter a compatibilidade binária é importante para garantir que os programas existentes possam continuar funcionando sem problemas à medida que as bibliotecas são atualizadas e melhoradas. No entanto, também pode ser um desafio, pois os desenvolvedores de bibliotecas devem ser cuidadosos para evitar alterações que quebrem a ABI. Isso muitas vezes requer um planejamento e design cuidadosos para permitir a evolução da biblioteca enquanto mantém a compatibilidade com as versões existentes.

Além disso, alguns sistemas possuem um recurso chamado versionamento de símbolos, que pode ajudar a manter a compatibilidade binária. O versionamento de símbolos permite que diferentes versões de funções ou variáveis existam na mesma biblioteca, cada uma com uma "versão" ou "símbolo" associado a ela. Isso permite que programas mais antigos que dependem de versões antigas de funções continuem a funcionar com a mesma biblioteca que também suporta versões mais recentes dessas funções.

5.4.2 Compatibilidade de código-fonte de bibliotecas

A compatibilidade de código-fonte de bibliotecas é um conceito chave em desenvolvimento de software e refere-se à capacidade de um programa se compilar e funcionar corretamente com diferentes versões de uma biblioteca. Especificamente, se a interface de programação de aplicação (API) da biblioteca permanecer consistente, o código-fonte de um programa que utiliza essa biblioteca não precisará ser alterado ou recompilado quando a biblioteca for atualizada.

A API de uma biblioteca é o conjunto de funções públicas, estruturas de dados e definições de tipo que a biblioteca oferece para serem usadas pelos programas. Quando um programa utiliza uma biblioteca, ele o faz através da API da biblioteca. O programa depende do fato de que as funções e estruturas de dados que ele utiliza da API se comportem de uma certa maneira e tenham uma certa estrutura.

Para garantir a compatibilidade de código-fonte, os desenvolvedores da biblioteca devem evitar fazer mudanças que alterem a API de maneira incompatível. Por exemplo, se uma função é removida da API ou se os argumentos de uma função são alterados, um programa que utiliza essa função não poderá mais ser compilado sem alterações no código-fonte.

Um exemplo poderia ser uma biblioteca que oferece uma função para ordenar uma lista de números. Se a função fosse definida inicialmente como `sort(int* array, int length)`, e mais tarde fosse alterada para `sort(int* array, int length, bool ascending)`, o código que usa essa função precisaria ser modificado para incluir o novo argumento `ascending`.

Embora seja às vezes necessário fazer mudanças que quebram a compatibilidade de código-fonte para corrigir erros ou adicionar novos recursos, geralmente é uma boa prática manter a compatibilidade sempre que possível. Fazer isso permite que os desenvolvedores de software que utilizam a biblioteca se beneficiem de melhorias e correções na biblioteca sem a necessidade de modificar e recompilar seus próprios programas.

Além disso, é uma prática comum introduzir novas funcionalidades de maneira a não quebrar a compatibilidade. Isso pode ser feito adicionando novas funções em vez de alterar as existentes, ou fornecendo valores padrão para novos argumentos em funções existentes.

Manter a compatibilidade de código-fonte é um componente chave na criação de bibliotecas de software robustas, usáveis e de longa duração.

6 CRIANDO E EXECUTANDO BINÁRIOS

No universo da programação de computadores, o processo de transformar código-fonte legível por humanos em um formato que a máquina pode entender e executar é fundamental para o desenvolvimento de software. Nos aprofundaremos nos detalhes desse processo crucial, focando na compilação e no linking de binários.

A compilação é o processo de transformar o código-fonte escrito em uma linguagem de programação de alto nível, como C ou Java, em código de máquina ou byte code, que pode ser executado diretamente pelo hardware ou interpretado por uma máquina virtual. O compilador é a ferramenta que realiza esse processo. Ele não apenas traduz o código-fonte para uma forma que a máquina possa entender, mas também verifica erros de sintaxe e de tipo, otimiza o código para desempenho, entre outras tarefas.

O linking, por outro lado, é o processo de combinar vários arquivos de código de máquina ou objetos em um único arquivo executável. O linker é responsável por essa tarefa, e ele resolve as referências entre diferentes partes do programa, vincula as chamadas de função às suas respectivas implementações e organiza a disposição dos dados na memória.

Ambos, compilação e linking, são etapas fundamentais no desenvolvimento de software, permitindo que o código-fonte seja transformado em programas executáveis que podem ser rodados em um computador. Entender esses processos é essencial para qualquer pessoa que esteja construindo ou depurando software, pois isso influencia diretamente a performance, a robustez e a portabilidade dos programas.

Neste capítulo, examinaremos em detalhes o que ocorre em cada uma dessas etapas, desde o momento em que o código-fonte é passado para o compilador até o ponto em que temos um programa binário pronto para ser executado. Vamos explorar os desafios e trade-offs envolvidos, bem como as várias estratégias usadas para otimizar esses processos. Então, prepare-se para uma viagem profunda através do mundo da compilação e linking de binários.

6.1 Compilação e linking

Vamos abordar o processo de compilação através de três etapas bases: pré-processamento, tradução e linking. A figura “Fluxo de compilação e linking” fornece em linhas gerais um resumo do processo:

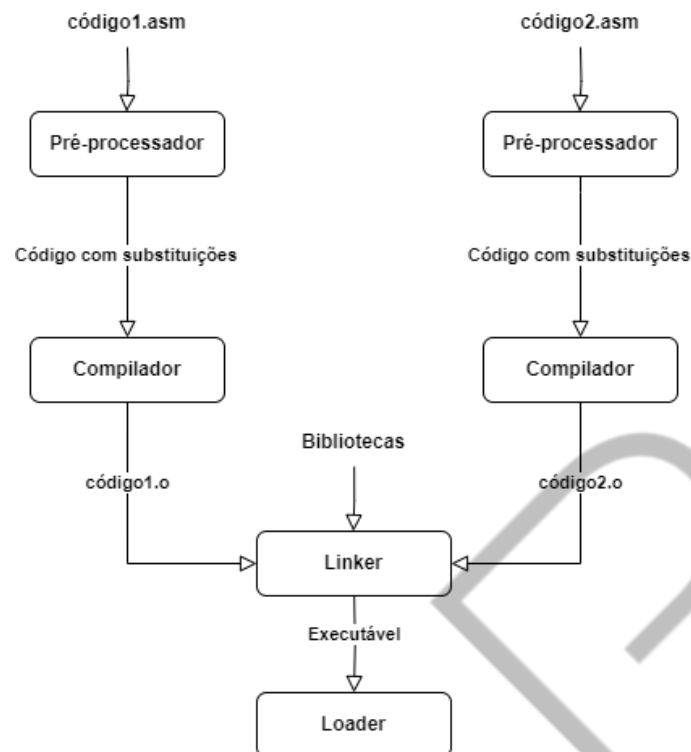


Figura 13 – Fluxo de compilação e linking
Fonte: Elaborada pelo autor (2023)

O pré-processador modifica o código-fonte do programa, resultando em outro conjunto de código na mesma linguagem. Normalmente, estas modificações envolvem substituições de uma cadeia de caracteres por outra.

O compilador, por sua vez, converte cada arquivo de código-fonte em um arquivo que contém instruções codificadas para a máquina, também conhecido como arquivo objeto. No entanto, esses arquivos não estão prontos para serem executados imediatamente, já que ainda não possuem as referências corretas para outros arquivos que foram compilados separadamente. Isso acontece nos casos onde as instruções se referem a dados ou a outras instruções que foram declaradas em diferentes arquivos.

O linker é responsável por estabelecer as conexões entre esses arquivos e criar um arquivo executável. Após essa etapa, o programa estará pronto para ser executado. Os linkers operam com arquivos-objeto que, geralmente, estão em formatos como ELF e COFF.

Por fim, o carregador (ou *loader*) processa um arquivo executável. Este tipo de arquivo, geralmente, apresenta uma estrutura bem definida e contém metadados. O carregador então preenche o espaço de endereçamento de um novo processo com as

instruções do arquivo executável, a pilha, os dados definidos globalmente e o código de tempo de execução fornecido pelo sistema operacional.

6.1.1 Pré-processador

Cada programa é inicialmente criado como um texto. A primeira fase da compilação é chamada de pré-processamento. Durante esta etapa, um programa especial avalia as diretivas do pré-processador encontradas no código-fonte do programa. De acordo com essas diretivas, são realizadas substituições textuais. Como resultado, obtemos um código-fonte modificado, sem diretivas do pré-processador, escrito na mesma linguagem de programação. Nesta seção, vamos discutir a utilização do macro processador NASM.

Uma das diretivas básicas do pré-processador é chamada de `%define`. Ela realiza uma substituição simples. Dado o código mostrado abaixo, um pré-processador substituirá "cat_count" por 42 sempre que encontrar tal substring no código-fonte do programa.

```
define_cat_count.asm
```

```
%define cat_count 42
```

```
mov rax, cat_count
```

Código-fonte 1 – Substituição simples
Fonte: Elaborada pelo autor (2023)

Para ver os resultados do pré-processamento para um arquivo `input_file.asm`, execute "`nasm -E file.asm`". Isso geralmente é muito útil para fins de depuração. Vamos ver o resultado abaixo para o arquivo do exemplo anterior.

```
define_cat_count_preprocessed.asm
```

```
%line 2+1 define_cat_count.asm
```

```
mov rax, 42
```

Código-fonte 2 – Resultado pré-processamento
Fonte: Elaborada pelo autor (2023)

Os comandos para declarar substituições são chamados de macros. Durante um processo chamado expansão de macro, suas ocorrências são substituídas por pedaços de texto. Os fragmentos de texto resultantes são chamados de instâncias de macro. No exemplo anterior, o número 42 na linha "mov rax, cat_count" é uma instância de macro. Nomes como "cat_count" são frequentemente referidos como símbolos do pré-processador.

É importante notar que o pré-processador conhece muito pouco, se é que conhece algo, sobre a sintaxe da linguagem de programação. Esta última define as construções válidas da linguagem. Por exemplo, o código mostrado no exemplo abaixo é correto. Não importa se nem 'a' nem 'b' isoladamente constituem uma construção válida em *assembly* (linguagem de montagem); contanto que o resultado das substituições seja sintaticamente válido, o compilador aceitará.

```
macro_asm_parts.asm
```

```
%define a mov rax,
```

```
%define b rbx
```

```
a b
```

Código-fonte 3 – Macros
Fonte: Elaborada pelo autor (2023)

Em outro exemplo, em linguagens de alto nível, uma instrução if tem a forma de "if (<expressão>) then <instrução> else <instrução>". Macros podem operar partes desta construção que, por si só, não são sintaticamente corretas (por exemplo, uma cláusula "else <instrução>" isolada). Desde que o resultado seja sintaticamente correto, o compilador não terá problemas.

Contrariamente, existem outros tipos de macros, especificamente, macros sintáticas, que estão vinculadas à estrutura da linguagem e operam com suas construções. Tais macros as modificam de maneira estruturada. Linguagens como LISP, OCaml e Scala usam macros sintáticas.

Por que usamos macros afinal? Além da automação, que veremos mais adiante, elas fornecem mnemônicos para partes do código. Para constantes, permitem-nos distinguir ocorrências de 42 usadas para contar gatos daquelas usadas para contar cães ou qualquer outra coisa. Caso contrário, certas modificações de programa seriam

mais dolorosas e propensas a erros, pois teríamos que tomar mais decisões baseadas no que esse número específico significa.

Para conjuntos de construções de linguagem, as macros nos fornecem uma certa automatização, assim como as sub-rotinas. As macros são expandidas no momento da compilação, enquanto as rotinas são executadas em tempo de execução. A escolha é sua.

Para *assembly*, nenhuma otimização é realizada nos programas. No entanto, em linguagens de alto nível, as pessoas usam variáveis constantes globais para esse fim. Um bom compilador substituirá suas ocorrências por seu valor. Um compilador ruim, no entanto, pode não estar ciente das otimizações, o que pode ser o caso ao programar microcontroladores ou aplicativos para sistemas operacionais exóticos. Nestes casos, as pessoas muitas vezes fazem o trabalho de um compilador, usando macros como na linguagem *assembly*.

Você pode pensar nas macros como uma metalinguagem de programação executada durante a compilação. Elas podem realizar cálculos bastante complexos e são limitadas de duas maneiras:

- Esses cálculos não podem depender da entrada do usuário (portanto, só podem operar constantes).
- Os ciclos podem ser executados apenas um número fixo de vezes. Isso significa que construções semelhantes a "while" são impossíveis de serem codificadas.

6.1.2 Tradução

Um compilador geralmente traduz o código-fonte de uma linguagem para outra. No caso da tradução de linguagens de programação de alto nível para código de máquina, este processo incorpora várias etapas internas. Durante esses estágios, gradualmente empurramos a Representação Intermediária (IR) do código em direção à linguagem alvo. Cada movimento da IR a aproxima mais da linguagem alvo. Logo antes de produzir o código *assembly*, a IR estará muito próxima do *assembly*, então podemos descarregar o *assembly* em uma listagem legível em vez de codificar instruções.

A tradução não é apenas um processo complexo, mas também perde informações sobre a estrutura do código-fonte, portanto, reconstruir o código de alto nível legível a partir do arquivo *assembly* é impossível.

Um compilador trabalha com entidades de código atômicas chamadas módulos. Um módulo geralmente corresponde a um arquivo de código-fonte (mas não a um arquivo de cabeçalho ou inclusão). Cada módulo é compilado independentemente dos outros módulos. O arquivo objeto é produzido a partir de cada módulo. Ele contém instruções codificadas em binário, mas geralmente não pode ser executado imediatamente. Existem várias razões para isso.

Por exemplo, o arquivo objeto é completado separadamente de outros arquivos, mas se refere a código e dados externos. Ainda não está claro se esse código ou dados residirão na memória, ou a posição do próprio arquivo objeto.

A tradução da linguagem *assembly* é bastante direta porque a correspondência entre os mnemônicos de *assembly* e as instruções de máquina é quase um para um. Além da resolução de rótulos, não há muito trabalho não trivial. Assim, por enquanto, nos concentraremos no seguinte estágio de compilação, ou seja, o link.

6.1.3 Linking

Usando o exemplo anterior para transformar o programa código.asm simples do código fonte para um programa executável podemos usar os seguintes comandos:

```
nasm -f elf64 -o codigo.o codigo.asm
```

```
ld -o codigo codigo.o
```

Comando de prompt 1 – Criando um arquivo objeto e linkando
Fonte: Elaborada pelo autor (2023)

Nesse exemplo usamos o NASM para produzir um arquivo objeto (object file). Especificamos seu formato, elf64, com a flag “-f”. E depois usamos um o programa ld, para produzir um arquivo que está pronto para ser executado.

O propósito de qualquer linker é fazer um object file executável ou compartilhado, a partir de um conjunto de uns realocáveis. Para que ele consiga fazer isso ele precisa realizar duas operações:

- Reallocation;
- Resolver os símbolos.

Como visto anteriormente um arquivo ELF possui um main header, que armazena as metainformações. Essas informações organizadas em seções descritas pela tabela de seções são acessíveis através do `readelf` (conforme será visto nas seções posteriores). Cada seção pode conter dados brutos que serão carregados na memória ou metadados formatados sobre outras seções, usados pelo *loader* (`.bss`), linker (reallocation) ou debuggers (`.line`).

Isso significa que o linker precisa mudar código de máquina compilado, preenchendo os endereços de memória absolutos em argumentos de instruções. Para conseguir isso, para cada símbolo, todas as referências a ele são lembradas na tabela de realocação. Assim que o linker compreende qual será seu verdadeiro endereço virtual, ele percorre a lista de ocorrências do símbolo e preenche as lacunas. Existe uma tabela de realocação separada para cada seção que necessita de uma.

6.1.4 Explicando gcc

O GCC (*GNU Compiler Collection*) é uma coleção de compiladores de código aberto para várias linguagens de programação, como C, C++, Objective-C, Fortran, Ada e Go, entre outras. O GCC é um projeto chave da GNU e é amplamente utilizado para desenvolver software de código aberto e software comercial.

Aqui estão alguns exemplos:

Se você quiser gerar um arquivo objeto (arquivo `.o`), pode fazer isso usando a opção `-c` do GCC. Um arquivo objeto é um arquivo binário que contém código de máquina, mas ainda não está completamente linkado e, portanto, não pode ser executado diretamente. Por exemplo:

```
gcc -c programa.c
```

Comando de prompt 2 – Compilando GCC
Fonte: Elaborada pelo autor (2023)

Neste caso, o GCC irá gerar um arquivo chamado "programa.o". Esse arquivo contém o código de máquina gerado a partir do seu arquivo-fonte "programa.c", mas ainda não está linkado a nenhum código de biblioteca ou outros arquivos objeto.

Se você tiver vários arquivos fonte e quiser compilar cada um deles em um arquivo objeto separado, pode fazer isso assim:

```
gcc -c arquivo1.c -o arquivo1.o
```

```
gcc -c arquivo2.c -o arquivo2.o
```

Comando de prompt 3 – Usando gcc
Fonte: Elaborada pelo autor (2023)

Depois de ter seus arquivos objeto, você pode linká-los em um único executável usando o GCC novamente, mas sem a opção "-c":

```
gcc arquivo1.o arquivo2.o -o programa
```

Comando de prompt 4 – Compilando arquivo objeto
Fonte: Elaborada pelo autor (2023)

Este último comando irá criar um executável chamado "programa" que é o resultado da linkagem dos arquivos objeto "arquivo1.o" e "arquivo2.o".

Compilando código-fonte com o GCC: A forma mais básica de usar o GCC é para compilar um único arquivo de código-fonte. Por exemplo, se você tiver um arquivo C chamado "programa.c", você pode compilá-lo usando o seguinte comando no terminal:

```
gcc programa.c -o programa
```

Comando de prompt 5 – Compilando arquivo c
Fonte: Elaborada pelo autor (2023)

Neste exemplo, "programa.c" é o arquivo de entrada (o código-fonte em C) e "programa" é o arquivo de saída (o programa executável).

Opções do compilador: O GCC tem muitas opções que permitem aos desenvolvedores controlar o processo de compilação. Por exemplo, a opção "-Wall" instrui o GCC a mostrar todos os avisos durante a compilação:

```
gcc -Wall programa.c -o programa
```

Comando de prompt 6 – Opções de compilação
Fonte: Elaborada pelo autor (2023)

Outra opção útil é "-g", que adiciona informações de depuração ao arquivo executável (útil para usar com ferramentas de depuração como o gdb):

Compilando múltiplos arquivos de origem: O GCC pode compilar vários arquivos de origem de uma só vez. Por exemplo, se você tiver dois arquivos de código-fonte, "programa1.c" e "programa2.c", você pode compilá-los juntos da seguinte maneira:

```
gcc -g programa.c programa.c -o programa
```

Comando de prompt 7 – Múltiplos arquivos
Fonte: Elaborada pelo autor (2023)

Linkagem de bibliotecas: O GCC também é responsável por ligar bibliotecas ao seu programa. Por exemplo, se você estiver usando a biblioteca matemática (libm), você precisa informar ao GCC para linká-la:

```
gcc programa.c -o programa -lm
```

Comando de prompt 8 – Biblioteca
Fonte: Elaborada pelo autor (2023)

Lembre-se, o GCC é uma ferramenta muito poderosa com muitos recursos e opções avançadas. É aconselhável se familiarizar com suas opções e funcionalidades para se tornar um desenvolvedor mais eficaz. A documentação oficial do GCC é uma excelente fonte de informações para isso.

6.2 Executando um arquivo binário

Executar arquivos binários no Linux pode parecer uma tarefa desafiadora à primeira vista, mas, na verdade, é um processo relativamente simples, que pode ser dividido em várias etapas. Vamos abordar essas etapas detalhadamente:

I. Tornando o arquivo binário executável:

No Linux, a permissão para executar um arquivo deve ser concedida explicitamente. Você pode verificar se um arquivo é executável usando o comando ls -l. Por exemplo:

```
ls -l meu_programa
```

Comando de prompt 9 – Listando arquivos
Fonte: Elaborada pelo autor (2023)

Isso listará o arquivo e suas permissões no formato -rwxrwxrwx. Cada trio de rwx representa as permissões de leitura (read), escrita (write) e execução (execute) para o proprietário do arquivo, o grupo do arquivo e outros usuários, respectivamente. Se a permissão de execução estiver definida (x), o arquivo é executável.

Para tornar um arquivo binário executável, você pode usar o comando `chmod`. Por exemplo:

```
chmod +x meu_programa
```

Comando de prompt 10 – Tornando o programa executável
Fonte: Elaborada pelo autor (2023)

Este comando adiciona a permissão de execução ao arquivo `meu_programa` para todos os usuários.

II. Executando o arquivo binário:

Agora que o arquivo é executável, você pode executá-lo de duas maneiras diferentes:

- Digitando o caminho para o arquivo binário no terminal. Se o arquivo estiver no diretório atual, você pode usar `./` para indicar isso. Por exemplo:

```
./meu_programa
```

Comando de prompt 11 – Executando o binário 1
Fonte: Elaborada pelo autor (2023)

Se o diretório do arquivo binário estiver no `PATH` do sistema, você pode executar o arquivo binário apenas digitando seu nome. Por exemplo:

```
meu_programa
```

Comando de prompt 12 – Executando o binário 2
Fonte: Elaborada pelo autor (2023)

O `PATH` do sistema é uma variável de ambiente que lista os diretórios onde o shell procurará por arquivos executáveis quando você digitar um comando. Você pode adicionar um diretório ao `PATH` com o comando `export`. Por exemplo:

```
export PATH=$PATH:/caminho/para/seu/diretório
```

Comando de prompt 13 – `PATH`
Fonte: Elaborada pelo autor (2023)

III. Passando argumentos para o arquivo binário:

Se o seu arquivo binário espera argumentos de linha de comando, você pode passá-los após o nome do arquivo. Por exemplo:

```
./meu_programa arg1 arg2 arg3
```

Comando de prompt 14 – Executando programa com argumentos
Fonte: Elaborada pelo autor (2023)

Neste exemplo, arg1, arg2 e arg3 são argumentos passados para o programa meu_programa.

IV. Redirecionando a entrada e saída do arquivo binário:

Você também pode redirecionar a entrada e a saída do seu programa. Por exemplo, para redirecionar a entrada do seu programa a partir de um arquivo chamado input.txt:

```
./meu_programa < input.txt
```

Comando de prompt 15 – Direcionando a entrada
Fonte: Elaborada pelo autor (2023)

Para redirecionar a saída do seu programa para um arquivo chamado output.txt:

```
./meu_programa > output.txt
```

Comando de prompt 16 – Redirecionando o output
Fonte: Elaborada pelo autor (2023)

V. Executando o arquivo binário em segundo plano:

Se o seu programa for um processo longo e você não quiser que ele bloqueie o terminal enquanto estiver em execução, você pode executá-lo em segundo plano adicionando um & no final do comando:

```
./meu_programa &
```

Comando de prompt 17 – Execução em segundo plano
Fonte: Elaborada pelo autor (2023)

Lembre-se de que a execução de arquivos binários pode ser uma operação perigosa se você não tiver certeza de onde o arquivo binário veio. Nunca execute arquivos binários de fontes desconhecidas ou não confiáveis.

7 ANÁLISE E DEBUGGING DE BINÁRIOS

Forneceremos uma introdução completa às técnicas e ferramentas necessárias para a análise e depuração de arquivos binários. Com foco em ferramentas como gdb, lldb, strace, objdump e readelf, exploraremos as complexidades e nuances associadas ao entendimento e solução de problemas em aplicações binárias.

A análise de binários é uma habilidade essencial para programadores, engenheiros de software e especialistas em segurança, pois fornece uma visão profunda do funcionamento interno de um programa compilado. Este entendimento pode ser crítico para resolver problemas de desempenho, corrigir bugs, entender problemas de segurança e muito mais.

Começamos com o “gdb” um poderoso depurador que permite que os programadores vejam o que acontece “por baixo do capô” de um programa enquanto ele está em execução. Eles são ferramentas essenciais para qualquer programador sério e, através de exemplos práticos, mostraremos como você pode usá-los para resolver problemas complexos em seus programas.

A seguir, discutiremos o “strace”, uma ferramenta útil que pode monitorar as chamadas de sistema que um programa realiza, o que pode ser muito útil para resolver problemas relacionados à interação do programa com o sistema operacional.

Avançamos então para o “objdump”, que permite que você veja a representação binária de um programa, incluindo o código de máquina, seções de código, cabeçalhos de arquivo e muito mais. Combinado com o conhecimento adquirido nas seções anteriores, isso permitirá que você tenha uma visão muito detalhada da estrutura e comportamento de um programa.

Finalmente, iremos explorar o “readelf”, que fornece uma maneira de inspecionar vários aspectos dos arquivos ELF, o formato padrão para arquivos binários em Linux.

Esperamos não apenas ensiná-lo a usar essas ferramentas, mas também a desenvolver uma compreensão mais profunda dos princípios fundamentais por trás delas. A depuração eficaz é tanto uma arte quanto uma ciência, dessa maneira, aperfeiçoar essa habilidade fará de você um programador melhor e mais eficaz.

7.1 Introdução ao gdb

O depurador é uma ferramenta muito potente à sua disposição. Ele permite executar programas passo a passo e monitorar o seu estado, incluindo os valores dos registradores e o conteúdo da memória. Usamos o depurador conhecido como gdb.

A depuração é um processo de encontrar falhas e estudar o comportamento de um programa. Para fazer isso, geralmente executamos etapas únicas, observando uma parte do estado do programa que é de nosso interesse. Também podemos executar o programa até que uma determinada condição seja atendida ou uma posição no código seja alcançada. Essa posição no código é chamada de ponto de interrupção.

Para executar o gdb e depurar um determinado executável basta dar o comando abaixo:

```
gdb programa
```

Comando de prompt 18 – 16 GDB
Fonte: Elaborada pelo autor (2023)

O gdb possui seu próprio sistema de comandos e a interação com ele ocorre por meio desses comandos. Portanto, sempre que o gdb é lançado e você vê seu prompt de comando (gdb), você pode digitar comandos e ele interagirá de acordo.

Você pode carregar um arquivo executável emitindo o comando "file" e em seguida digitando o nome do arquivo, ou passando-o como um argumento.

```
(gdb) file programa
```

Comando de prompt 19 – Importando o programa
Fonte: Elaborada pelo autor (2023)

A tecla <tab> tem a função de fornecer sugestões de autocompletar no prompt de comando gdb. Muitos comandos também possuem atalhos.

Os dois comandos mais importantes são:

- quit para encerrar o gdb.
- help cmd para mostrar a ajuda para o comando cmd.

O arquivo ~/.gdbinit armazena comandos que serão executados automaticamente quando o gdb iniciar. Tal arquivo também pode ser criado no diretório atual, mas por razões de segurança, esse recurso é desativado por padrão.

Por padrão, o gdb usa a sintaxe de montagem AT&T. No entanto, neste material, adotamos a sintaxe Intel; para alterar as preferências padrão do gdb em relação à sintaxe de montagem, adicione a seguinte linha ao arquivo ~/.gdbinit:

```
set disassembly-flavor intel
```

Comando de prompt 20 – Configurando o disassembly
Fonte: Elaborada pelo autor (2023)

Outros comandos úteis incluem:

- run para iniciar a execução do programa.
- break x para criar um ponto de interrupção próximo ao rótulo x.
- Ao executar run ou continue, vamos parar no primeiro ponto de interrupção alcançado, permitindo-nos examinar o estado do programa.
- break *address para colocar um ponto de interrupção em um endereço especificado.
- continue para continuar a execução do programa.
- stepi ou si para avançar uma instrução.
- ni ou nexti também executam uma instrução, mas não entram em funções se a instrução for call. Em vez disso, eles permitem que a função chamada termine e interrompem na próxima instrução.

Vamos usar de exemplo o seguinte programa:


```
section .data
codes:
db '0123456789ABCDEF'
section .text
global _start
_start:
mov rax, 0x1122334455667788
mov rdi, 1
mov rdx, 1
mov rcx, 64
.loop:
push rax
sub rcx, 4
sar rax, cl
and rax, 0xf
lea rsi, [codes + rax]
mov rax, 1
push rcx
syscall
pop rcx
Chapter 18 ■ Appendix A. Using gdb
400
pop rax
test rcx, rcx
jnz .loop
mov rax, 60 ; invoke 'close' system call
xor rdi, rdi
syscall
```

Código-fonte 4 – Programa print RAX
Fonte: Elaborada pelo autor (2023)

Esse programa simples faz o print do registrador rax na tela.

Para analisa-lo vamos primeiro compila-lo usando nasm:

```
$ nasm -o print_rax.o -f elf64 print_rax.asm
```

```
$ ld -o print_rax print_rax.o
```

```
gdb print_rax
```

```
(gdb) file print_rax
```

Comando de prompt 21 – Exemplo 1
 Fonte: Elaborada pelo autor (2023)

Se quisermos colocar um breakpoint podemos fazer o seguinte:

(gdb) break _start

(gdb) start

Comando de prompt 22 – Breakpoint
 Fonte: Elaborada pelo autor (2023)

O programa irá iniciar e parar no breakpoint que colocamos.

Se usarmos os comandos layout asm e layout regs, podemos mudar para o modo pseudo gráfico:

The screenshot shows the GDB 'Layouts' window. The top panel, titled 'Register group: general', displays the current values of various registers in hexadecimal. The bottom panel shows the assembly code for the program, with the current instruction highlighted.

Register	Value	Register	Value
rax	0x1122334455667788	rbx	0x0
rcx	0x0	rdx	0x0
rsi	0x0	rdi	0x0
rbp	0x0	rsp	0x7fffffffdf40
r8	0x0	r9	0x0
r10	0x0	r11	0x0
r12	0x0	r13	0x0
r14	0x0	r15	0x0
rip	0x40100a	eflags	0x202
cs	0x33	ss	0x2b
ds	0x0	es	0x0
fs	0x0	gs	0x0

Address	Disassembly
0x401000 <_start>	movabs \$0x1122334455667788,%rax
0x40100a <_start+10>	mov \$0x1,%edi
0x40100f <_start+15>	mov \$0x1,%edx
0x401014 <_start+20>	mov \$0x40,%ecx
0x401019 <_start.loop>	push %rax
0x40101a <_start.loop+1>	sub \$0x4,%rcx
0x40101e <_start.loop+5>	sar %cl,%rax
0x401021 <_start.loop+8>	and \$0xf,%rax
0x401025 <_start.loop+12>	lea 0x402000(%rax),%rsi
0x40102c <_start.loop+19>	mov \$0x1,%eax
0x401031 <_start.loop+24>	push %rcx
0x401032 <_start.loop+25>	syscall
0x401034 <_start.loop+27>	pop %rcx
0x401035 <_start.loop+28>	pop %rax

Figura 14 – Layouts

Fonte: Elaborada pelo autor (2023)

Na figura “Layouts”, o painel de cima são os registradores e os valores em hexadecimal presentes dentro deles. Já o painel de baixo mostra o código no dissasembler. Somente uma das janelas pode ser selecionada por vez para mudar a seleção pressiona-se CTRL+X e CTRL+O.

As setas servem para ir para cima e para baixo na janela. O comando print /FMT <val> fornece uma forma de buscar por um valor no registrador ou memória e x /FMT <endereço> é usado para verificar o conteúdo de um endereço de memória.

Para verificar o que está contido em um registrador RAX podemos usar o comando `print $rax`.

Outros comandos interessantes são:

- Exibindo o primeiro caractere de codes:

```
(gdb) print /c codes
```

```
$2 = '0'
```

Comando de prompt 23 – Print de caracteres
Fonte: Elaborada pelo autor (2023)

- Desassemble de uma instrução no endereço `_start`:

```
(gdb) x /i &_start
```

```
0x4000b0 <_start>: movabs rax,0x1122334455667788
```

Comando de prompt 24 – Desassemble uma instrução
Fonte: Elaborada pelo autor (2023)

- Desassemble da instrução atual:

```
(gdb) x /i $rip
```

```
=> 0x4000e9 <_start.loop+32>: jne 0x4000c9 <_start.loop>
```

Comando de prompt 25 – Desassemble instrução atual
Fonte: Elaborada pelo autor (2023)

- Verificando o conteúdo de codes. A parte `/FMT` do comando `x` pode começar com a contagem de elementos. No nosso caso, `/12cb` significa "12 caracteres, cada um com um byte".

```
(gdb) x /12cb &codes
```

```
0x6000f8 <codes>: 48 '0' 49 '1' 50 '2' 51 '3' 52 '4' 53 '5' 54 '6'
```

```
55 '7'
```

```
0x600100: 56 '8' 57 '9' 65 'A' 66 'B'
```

Comando de prompt 26 – Verificando o conteúdo de codes
Fonte: Elaborada pelo autor (2023)

- Examinando os 8 bytes superiores da pilha:

```
(gdb) x /x $rsp
```

0x7fffffffdf90: 0x01

Comando de prompt 27 – Examinando os 8 bytes
Fonte: Elaborada pelo autor (2023)

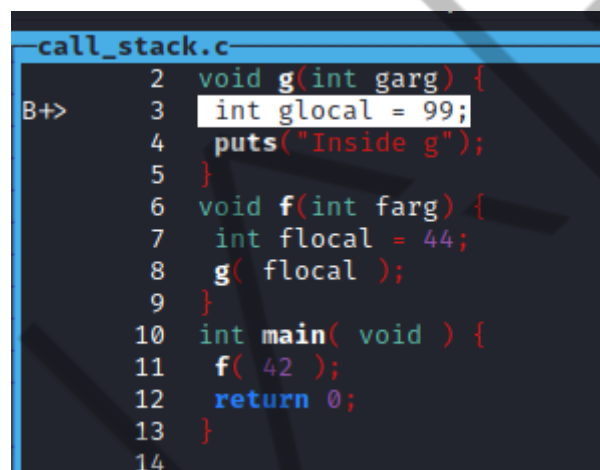
- Examinando o segundo qword armazenado na pilha:

(gdb) x/x \$rsp+8

0x7fffffffdf98: 0xc1

Comando de prompt 28 – Segundo qword
Fonte: Elaborada pelo autor (2023)

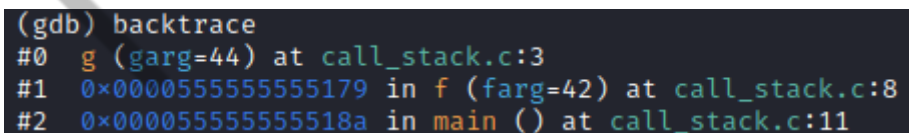
Por fim, é interessante utilizar a opção layout src para ver o código fonte quando trabalhamos com linguagens de alto nível como C. Poderemos por meio dele ver um código base que o gdb gera para tentarmos entender o programa melhor.



```
call_stack.c
2 void g(int garg) {
3     int glocal = 99;
4     puts("Inside g");
5 }
6 void f(int farg) {
7     int flocal = 44;
8     g( flocal );
9 }
10 int main( void ) {
11     f( 42 );
12     return 0;
13 }
14
```

Comando de prompt 29 – Layou src
Fonte: Elaborada pelo autor (2023)

Usando o comando backtrace também conseguimos ver a última função que foi chamada.



```
(gdb) backtrace
#0  g (garg=44) at call_stack.c:3
#1  0x0000555555555179 in f (farg=42) at call_stack.c:8
#2  0x000055555555518a in main () at call_stack.c:11
```

Comando de prompt 30 – backtrace
Fonte: Elaborada pelo autor (2023)

7.2 strace

Strace é uma poderosa ferramenta de linha de comando usada em sistemas Unix e Linux para depurar e solucionar problemas em programas. Ele fornece um

mecanismo para rastrear chamadas de sistema e sinais - as principais interações entre programas do espaço do usuário e o kernel.

Chamadas de sistema são o principal método usado por um programa de usuário para solicitar um serviço do kernel do sistema operacional. Elas fornecem a interface entre um processo e o sistema operacional, permitindo que um processo solicite um serviço que só pode ser executado pelo kernel. Exemplos de tais serviços incluem manipulação de arquivos, controle de processos, rede e muito mais.

Quando você executa um programa com `strace`, ele exibe uma lista de todas as chamadas de sistema feitas pelo programa. Cada linha da saída representa uma chamada de sistema, e mostra o nome da chamada de sistema, seus argumentos e seu valor de retorno.

Usar `strace` pode ser muito útil em vários cenários. Por exemplo, se um programa não está se comportando como esperado, `strace` pode ser usado para ver quais chamadas de sistema estão falhando e, possivelmente, entender por quais motivos. Isso pode ajudar você ou os desenvolvedores do software a corrigir o problema. Também pode ser usado para entender como os programas funcionam, observando quais chamadas de sistema eles usam.

Para usar `strace`, você simplesmente prefixa seu comando com `strace`. Por exemplo, `strace ls` executaria o comando `ls` sob `strace`, imprimindo todas as chamadas de sistema feitas por `ls`.

A saída pode ser bastante detalhada, especialmente para programas maiores, mas pode ser filtrada com base em suas necessidades usando várias opções. Por exemplo, você pode filtrar chamadas de sistema por nome, seguir processos filhos ou gravar a saída em um arquivo para análise posterior.

Vale a pena notar que `strace` é uma ferramenta passiva, ela não modifica a execução do programa além de pausá-lo momentaneamente para registrar uma chamada de sistema. Isso o torna seguro para uso na maioria dos cenários, mas seu uso vem com algum impacto no desempenho devido ao trabalho adicional de registrar chamadas de sistema.

Exemplo:

```
$ strace cat /dev/null
```

```
>execve("/usr/bin/cat", ["cat", "/dev/null", "-o", "text"], 0x7ffd72e1f468 /* 55 vars */) = 0
>brk(NULL) = 0x556e3fa09000
```

Comando de prompt 31 – strace
 Fonte: Elaborada pelo autor (2023)

A primeira parte do output está chamando a função `execve()` para executar um novo programa (`/usr/bin/cat`). O comando é seguido de uma inicialização da memória `brk(NULL)`.

A próxima parte do output está relacionada a carregar as bibliotecas compartilhadas:

```
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=86966, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 86966, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe860adb000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0Ps\2\0\0\0\0"... , 832) = 832
```

Outra parte importante que podemos visualizar no output é a região final do código:

```
newfstatat(1, "", {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH)
= 0
openat(AT_FDCWD, "/dev/null", O_RDONLY) = 3
newfstatat(3, "", {st_mode=S_IFCHR|0666, st_rdev=makedev(0x1, 0x3), ...}, AT_EMPTY_PATH)
= 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7f2027b7b000
read(3, "", 131072) = 0
munmap(0x7f2027b7b000, 139264) = 0
close(3) = 0
close(1) = 0
```

```
close(2)                = 0
exit_group(0)           = ?
+++ exited with 0 +++
```

Código-fonte 5 – strace cat
Fonte: Elaborada pelo autor (2023)

Essa parte mostra o comando em execução. Primeiro veja a call `openat()` que abre o arquivo. O 3 é o resultado da operação que significa que foi bem sucedida. Abaixo disso pode-se ver que o comando `cat` le o conteúdo `/dev/null` (função `read()`). Como não tem nada para ler ele fecha o arquivo e depois termina com o `exit_group()`.

7.3 Objdump e readelf

7.3.1 Objdump

Objdump é uma ferramenta extremamente útil que faz parte do conjunto de ferramentas binárias GNU (GNU Binutils). Ela é usada para analisar, examinar e depurar arquivos de objeto em diversas formas. Arquivos de objeto são o resultado da compilação de código-fonte e são o que a fase de vinculação (linking) usa para criar um executável ou uma biblioteca.

Aqui estão algumas das funções mais utilizadas de ``objdump``:

- **Desmontagem de código binário:** Uma das funções mais comuns de ``objdump`` é a desmontagem de código binário. Isso significa que ele pode pegar um arquivo de objeto ou binário e traduzir o código de máquina de volta em uma forma mais legível de *assembly*. Este recurso é frequentemente usado em engenharia reversa para analisar o que um binário desconhecido ou suspeito está fazendo. Pode ser acessado por meio da opção ``-d`` ou ``--disassemble``.
- **Exibição de informações do cabeçalho:** ``objdump`` pode ser usado para exibir informações do cabeçalho do arquivo de objeto, incluindo a arquitetura para a qual o binário foi construído, o ponto de entrada do programa, seções do arquivo de objeto e símbolos. Isso é útil para entender a estrutura de alto nível do binário. Isso pode ser acessado através das opções ``-h`` ou ``--headers``.
- **Exibição de conteúdo da seção:** Além de exibir as informações de cabeçalho das seções, ``objdump`` também pode ser usado para exibir o conteúdo dessas

seções. Pode ser usado para ver o conteúdo das seções de texto, dados e muitos outros. Isso é acessado através da opção `-s` ou `--full-contents`.

- **Exibição de tabelas de símbolos:** `objdump` também pode ser usado para exibir as tabelas de símbolos de um arquivo de objeto. A tabela de símbolos é onde informações sobre símbolos (como funções e variáveis) são armazenadas em um arquivo de objeto. Isso é útil para entender quais símbolos estão presentes em um arquivo de objeto, e pode ajudar na depuração. Isso é acessado através da opção `-t` ou `--syms`.

Para usar `objdump`, você geralmente invoca o comando `objdump`, seguido por opções que especificam o que você quer que ele faça, seguido pelo nome do arquivo de objeto que você quer que ele processe. Por exemplo, `objdump -d myprog` desmontará o código no arquivo de objeto `myprog`, conforme pode ser visto abaixo:

Desktop/call_stack: file format elf64-x86-64

Disassembly of section .init:

0000000000001000 <_init>:

```
1000: 48 83 ec 08      sub    $0x8,%rsp
1004: 48 8b 05 c5 2f 00 00 mov    0x2fc5(%rip),%rax    # 3fd0 <__gmon_start__@Base>
100b: 48 85 c0          test   %rax,%rax
100e: 74 02            je     1012 <_init+0x12>
1010: ff d0            call   *%rax
1012: 48 83 c4 08      add    $0x8,%rsp
1016: c3              ret
```

Disassembly of section .plt:

0000000000001020 <puts@plt-0x10>:

```
1020: ff 35 ca 2f 00 00      push   0x2fca(%rip)        # 3ff0
<_GLOBAL_OFFSET_TABLE_+0x8>
1026: ff 25 cc 2f 00 00      jmp     *0x2fcc(%rip)      # 3ff8
<_GLOBAL_OFFSET_TABLE_+0x10>
102c: 0f 1f 40 00          nopl    0x0(%rax)
```

0000000000001030 <puts@plt>:

```
1030: ff 25 ca 2f 00 00      jmp     *0x2fca(%rip)      # 4000 <puts@GLIBC_2.2.5>
1036: 68 00 00 00 00      push    $0x0
103b: e9 e0 ff ff          jmp     1020 <_init+0x20>
```

Disassembly of section .plt.got:

0000000000001040 <__cxa_finalize@plt>:

```
1040: ff 25 9a 2f 00 00      jmp     *0x2f9a(%rip)      # 3fe0 <__cxa_finalize@GLIBC_2.2.5>
1046: 66 90              xchg    %ax,%ax
```

Disassembly of section .text:

0000000000001050 <_start>:

```
1050: 31 ed            xor     %ebp,%ebp
```

Código-fonte 6 – objdump
Fonte: Elaborada pelo autor (2023)

7.3.2 Readelf

`readelf` é uma ferramenta de diagnóstico que faz parte do conjunto de ferramentas binárias GNU (GNU Binutils) e é usada para exibir informações detalhadas sobre arquivos binários no formato ELF (Executable and Linkable Format). Este é o

formato padrão para executáveis, arquivos de objeto, bibliotecas compartilhadas e dumps de núcleo no Linux e em muitos outros sistemas operacionais baseados em Unix.

As informações que o ``readelf`` pode exibir incluem:

1. **Cabeçalhos ELF:** Este é o cabeçalho principal do arquivo e contém informações globais sobre o arquivo inteiro, como o tipo do arquivo (por exemplo, executável, arquivo de objeto relocável, biblioteca compartilhada, etc.), a arquitetura da máquina para a qual o arquivo é destinado, o ponto de entrada do programa e muito mais. O comando para exibir o cabeçalho ELF é ``readelf -h``.
2. **Cabeçalhos do programa:** Estes são metadados usados pelo sistema operacional para executar o binário. Cada cabeçalho de programa descreve um segmento do binário, como código, dados, informações de pilha e muito mais. O comando para exibir os cabeçalhos do programa é ``readelf -l``.
3. **Cabeçalhos de seção:** Estes são metadados usados durante a vinculação para combinar arquivos de objeto. Cada cabeçalho de seção descreve uma seção do arquivo de objeto, que pode conter código, dados, informações de depuração e muito mais. O comando para exibir os cabeçalhos de seção é ``readelf -S``.
4. **Tabelas de símbolos:** A tabela de símbolos de um binário lista os nomes de todas as funções e variáveis globais no binário, juntamente com informações sobre onde estão localizados. O comando para exibir a tabela de símbolos é ``readelf -s``.
5. **Informações de depuração:** Se um binário foi construído com informações de depuração, ``readelf`` pode exibir essas informações, que podem incluir o nome do arquivo de origem, números de linha e muito mais. O comando para exibir informações de depuração é ``readelf --debug-dump``.
6. **Tabelas de Relocalização:** Estas tabelas contêm informações que o vinculador usa para atualizar referências a símbolos quando o binário é carregado na memória. O comando para exibir tabelas de relocalização é ``readelf -r``.

8 PACOTES E DISTRIBUIÇÃO DE BINÁRIOS

8.1 Bibliotecas estáticas e dinâmicas

Praticamente todo programa utiliza código proveniente de bibliotecas, que podem ser de dois tipos: estáticas e dinâmicas.

As bibliotecas estáticas consistem em diversos arquivos de objeto realocáveis que são vinculados ao programa principal, fundindo-se ao arquivo executável resultante. No universo do Windows, esses arquivos possuem a extensão `.lib`. No ambiente Unix, esses são arquivos `.o` ou arquivos `.a` que contêm vários arquivos `.o`.

Por outro lado, as bibliotecas dinâmicas, também conhecidas como arquivos de objeto compartilhados, constituem um dos três tipos de arquivos de objeto que definimos anteriormente. Elas são vinculadas ao programa durante sua execução. Esses são os famosos arquivos `.dll` no Windows, e arquivos com extensão `.so` (objetos compartilhados) no Unix.

Embora as bibliotecas estáticas sejam apenas executáveis parcialmente prontos sem pontos de entrada, as bibliotecas dinâmicas têm algumas diferenças que vamos explorar agora.

As bibliotecas dinâmicas são carregadas quando necessárias. Sendo arquivos de objeto autônomos, elas possuem todos os tipos de metainformações sobre o código que fornecem para uso externo. Essa informação é utilizada por um carregador para determinar os endereços exatos de funções e dados exportados.

Essas bibliotecas podem ser fornecidas separadamente e atualizadas de forma independente, o que tem seus prós e contras. Enquanto o fabricante da biblioteca pode fornecer correções de bugs, ele também pode quebrar a compatibilidade retroativa ao alterar, por exemplo, argumentos de funções, o que seria como enviar uma mina de ação retardada.

Um programa pode trabalhar com uma quantidade ilimitada de bibliotecas compartilhadas, que devem ser carregáveis em qualquer endereço. Caso contrário, elas ficariam presas ao mesmo endereço, o que nos colocaria na mesma situação que temos ao tentar executar vários programas no mesmo espaço de endereço de memória física. Existem duas formas de contornar isso:

- I. Podemos realizar uma realocação em tempo de execução, quando a biblioteca está sendo carregada. No entanto, isso nos priva de um recurso muito atraente: a possibilidade de reutilizar o código da biblioteca na memória física sem

duplicação quando vários processos a estão utilizando. Se cada processo realiza a realocação da biblioteca para um endereço diferente, as páginas correspondentes tornam-se remendadas com diferentes valores de endereço e, portanto, tornam-se diferentes para diferentes processos. Em efeito, a seção data seria realocada de qualquer maneira por sua natureza mutável. Renunciando variáveis globais nos permite descartar tanto a seção quanto a necessidade de realocá-la. Outro problema é que a seção .text deve ser mantida gravável para realizar sua modificação durante o processo de realocação, introduzindo certos riscos de segurança, pois deixa sua modificação possível por código malicioso. Além disso, alterar o .text de cada objeto compartilhado quando várias bibliotecas são necessárias para um executável ser executado pode consumir muito tempo.

- II. Podemos escrever o Código Independente de Posição (PIC). Agora é possível escrever um código que pode ser executado independente de onde esteja na memória. Para isso, temos que eliminar completamente os endereços absolutos. Hoje em dia, os processadores suportam endereçamento relativo a rip, como `mov rax, [rip + 13]`. Este recurso facilita a geração do PIC. Esta técnica permite o compartilhamento da seção .text. Atualmente, os programadores são fortemente encorajados a usar PIC em vez de realocações.

As bibliotecas dinâmicas economizam espaço em disco e memória. Vale lembrar que as páginas podem ser marcadas como privadas ou compartilhadas entre vários processos. Quando uma biblioteca é utilizada por múltiplos processos, a maior parte dela não é duplicada na memória física.

8.2 Criando .deb e .rpm

8.2.1 .deb

Um arquivo .deb é um pacote de software para sistemas operacionais baseados em Debian. Ele contém todos os arquivos necessários para instalar um software específico em um sistema, bem como informações sobre as dependências do software (outros softwares de que ele precisa para funcionar corretamente) e como instalá-lo e removê-lo do sistema.

Estrutura de um arquivo .deb

Os arquivos .deb são arquivos arquivados que contêm três partes principais:

- I. **debian-binary:** Um arquivo de texto que indica a versão do formato do pacote .deb.
- II. **control.tar.gz:** Um arquivo comprimido que contém metadados sobre o pacote, como o nome do pacote, a versão do pacote, o fabricante, a descrição e as dependências. Ele também inclui scripts que são executados antes e depois da instalação e remoção do pacote.
- III. **data.tar.gz:** Este é outro arquivo comprimido que contém os arquivos reais do software que será instalado. Esses arquivos são extraídos para o sistema de arquivos do sistema durante a instalação.

Instalação de um arquivo .deb

Os arquivos .deb podem ser instalados usando a ferramenta de linha de comando `dpkg` fornecida pelo sistema operacional. O comando a seguir instala um pacote .deb:

```
dpkg -i package.deb
```

Comando de prompt 32 – Instalando um arquivo deb
Fonte: Elaborada pelo autor (2023)

Em que `package.deb` é o nome do arquivo .deb que você deseja instalar.

Se o pacote tiver dependências que não estão instaladas, o `dpkg` informará sobre elas, mas não as instalará. Nesse caso, você pode usar o comando `apt-get` para instalar as dependências e o pacote:

```
sudo apt install -f
```

Comando de prompt 33 – Instalando dependências
Fonte: Elaborada pelo autor (2023)

Remoção de um pacote .deb

Um pacote .deb instalado pode ser removido do sistema usando o comando `dpkg` da seguinte maneira:

```
dpkg -r package
```

Comando de prompt 34 – Removendo um pacote deb
Fonte: Elaborada pelo autor (2023)

Em que `package` é o nome do pacote que você deseja remover.

Criação de um pacote .deb

```
mkdir helloworld && mkdir helloworld/DEBIAN
```

```
mkdir -p helloworld/usr/local/bin cp /usr/local/bin/helloworld.sh helloworld/usr/local/bin/
```

Comando de prompt 35 – Criando um pacote
Fonte: Elaborada pelo autor (2023)

Crie um pacote de controle com um editor de texto e insira as seguintes linhas:

Package: helloworld

Version: 0.2

Maintainer: Pr4na

Architecture: all

Description: hello world

Comando de prompt 36 – Criando o pacote de controle
Fonte: Elaborada pelo autor (2023)

Crie o pacote .deb:

```
dpkg-deb --build helloworld
```

Comando de prompt 37– Gerando o deb
Fonte: Elaborada pelo autor (2023)

8.2.2 .rpm

Um arquivo .rpm é um pacote de software para sistemas operacionais baseados em Red Hat. Ele contém todos os arquivos necessários para instalar um software específico em um sistema, além de informações sobre as dependências do software (outros softwares dos quais ele precisa para funcionar corretamente) e instruções sobre como instalá-lo e removê-lo do sistema.

Estrutura de um arquivo .rpm

Os arquivos .rpm têm uma estrutura complexa que inclui a seguinte:

1. **Cabeçalho RPM:** Inclui metadados sobre o pacote, como o nome do pacote, a versão do pacote, o fabricante, a descrição e as dependências. O cabeçalho também contém informações sobre a instalação e remoção do pacote.
2. **Assinatura:** Uma assinatura digital que verifica a autenticidade e a integridade do pacote.
3. **Arquivo: CPIO:** Um arquivo que contém os arquivos reais do software que serão instalados. Esses arquivos são extraídos para o sistema de arquivos durante a instalação.

Instalação de um arquivo .rpm

Os arquivos .rpm podem ser instalados usando a ferramenta de linha de comando rpm fornecida pelo sistema operacional. O comando a seguir instala um pacote .rpm:

```
rpm -i package.rpm
```

Comando de prompt 38 – Instalando rpm
Fonte: Elaborada pelo autor (2023)

Em que package.rpm é o nome do arquivo .rpm que você deseja instalar.

Se o pacote tiver dependências que não estão instaladas, o rpm não as instalará. Nesse caso, você pode usar o comando yum ou dnf (no Fedora) para instalar as dependências e o pacote:

```
sudo yum localinstall package.rpm
```

ou

```
sudo dnf install package.rpm
```

Comando de prompt 39 – Instalando dependências
Fonte: Elaborada pelo autor (2023)

Remoção de um pacote .rpm

Um pacote .rpm instalado pode ser removido do sistema usando o comando rpm da seguinte maneira:

```
rpm -e package
```

Comando de prompt 40 – Removendo pacotes
Fonte: Elaborada pelo autor (2023)

Onde package é o nome do pacote que você deseja remover.

Criação de um pacote .rpm

Os desenvolvedores que desejam distribuir seu software como um pacote .rpm precisam criar um arquivo .rpm para o seu software. Isso geralmente envolve a escrita de um arquivo spec que contém instruções sobre como construir o pacote, e então usar uma ferramenta como rpmbuild para criar o arquivo .rpm.

Crie um script simples (helloworld.sh):

```
$ cat << EOF >> hello.sh
#!/bin/sh
echo "Hello world"
EOF
```

Código-fonte 7 – Script simples
Fonte: Elaborada pelo autor (2023)

Coloque o script em um diretório:

```
$ mkdir hello-0.0.1
$ mv hello.sh hello-0.0.1
$ tar --create --file hello-0.0.1.tar.gz hello-0.0.1
$ mv hello-0.0.1.tar.gz SOURCES
$ rpmdev-newspec hello
$ rpmbuild -bs ~/rpmbuild/SPECS/hello.spec
```

Comando de prompt 41 – Gerando pacotes rpm
Fonte: Elaborada pelo autor (2023)

REFERÊNCIAS

IBM. Executable Linking Format (ELF). 2021. Disponível em: <[Executable and linking format \(ELF\) - IBM Documentation](#)>. Acesso em: 10 jul. 2023.

TEXAS INSTRUMENTS. Common Object File Format. 2009. Disponível em: <<https://www.ti.com/lit/an/spraa08/spraa08.pdf?ts=1688953493796>>. Acesso em: 20 jul. 2023.

EMEND